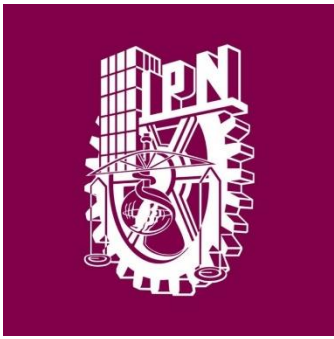




Instituto Politécnico
Nacional
Escuela Superior de
Computo



Profesor: Cortes Galicia Jorge

Materia: Sistemas Operativos

Practica 7

Integrantes:

Vázquez Blancas Cesar Said

Diaz Torres Jonathan Samuel

Aparicio Ponce Roberto Manuel

Corona Hernández Martin Rafael

Introducción Teórica

La **Comunicación Interprocesos (IPC)** se refiere al conjunto de mecanismos que permiten a los procesos en un sistema operativo compartir información y coordinar su ejecución. Tanto en **Linux** como en **Windows**, estos mecanismos son esenciales para construir sistemas multitarea y distribuidos donde los procesos interactúan entre sí.

Objetivos de la IPC

- **Compartir datos:** Permitir el intercambio de información entre procesos, ya sea en el mismo sistema o entre sistemas distintos.
- **Sincronización:** Coordinar la ejecución de procesos para evitar conflictos y asegurar el acceso ordenado a recursos compartidos.
- **Notificación:** Informar a un proceso cuando otro realiza una acción específica.
- **Optimización de recursos:** Minimizar el uso de memoria y CPU al compartir datos entre procesos.

IPC en Linux

1. Basado en Archivos:

- **Pipes:** Comunicación unidireccional entre procesos relacionados.
- **FIFO (Named Pipes):** Similar, pero entre procesos no relacionados.

2. Basado en Memoria:

- **Memoria Compartida:** Procesos acceden a la misma región de memoria (rápido).
- **Semáforos:** Sincronización para evitar conflictos en recursos compartidos.
- **Colas de Mensajes:** Intercambio de mensajes estructurados.

3. Otros:

- **Sockets:** Comunicación local o remota.
 - **Señales:** Notificación de eventos entre procesos.
-

IPC en Windows

1. Basado en Archivos:

- **Named Pipes:** Comunicación bidireccional entre procesos locales o remotos.
- **Mailslots:** Mensajes unidireccionales breves.

2. Basado en Memoria:

- **Memoria Compartida:** Uso de mapeo de archivos para compartir áreas de memoria.
- **Mutex y Semáforos:** Sincronización de acceso.

3. Otros:

- **Sockets:** Comunicación en red con WinSock.
- **RPC:** Llamadas remotas a procedimientos.
- **COM:** Interacción entre objetos y procesos.

1. A través de la ayuda en línea que proporciona Linux, investigue el funcionamiento de las llamadas al sistema: pipe(), shmget(), shmat(). Explique los argumentos y retorno de la función.

1. pipe()

Crea un canal unidireccional de comunicación (tubería) entre procesos relacionados.

- **Prototipo:**

```
int pipe(int pipefd[2]);
```

- **Argumentos:**

- pipefd: Un arreglo de dos enteros.
 - pipefd[0]: Descriptor de archivo para leer desde la tubería.
 - pipefd[1]: Descriptor de archivo para escribir en la tubería.

- **Retorno:**

- **0:** Si se creó exitosamente.
 - **-1:** En caso de error, se establece errno.
-

2. shmget()

Crea o accede a un segmento de memoria compartida.

- **Prototipo:**

```
int shmget(key_t key, size_t size, int shmflg);
```

- **Argumentos:**

- key: Una clave única que identifica el segmento (generalmente generado con ftok()).
- size: Tamaño del segmento de memoria en bytes.
- shmflg: Flags de control.
 - IPC_CREAT: Crea el segmento si no existe.
 - IPC_EXCL: Falla si el segmento ya existe (usado junto con IPC_CREAT).
 - Permisos (e.g., 0666 para lectura/escritura).

- **Retorno:**
 - Un identificador positivo del segmento en caso de éxito.
 - **-1:** En caso de error, se establece errno.
-

3. shmat()

Adjunta un segmento de memoria compartida al espacio de direcciones del proceso.

- **Prototipo:**

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
```

- **Argumentos:**
 - shmid: Identificador del segmento de memoria (devuelto por shmget).
 - shmaddr: Dirección en la cual se debe adjuntar (usualmente NULL para permitir que el sistema decida).
 - shmflg: Flags de control.
 - SHM_RDONLY: Adjunta en modo solo lectura.
 - 0: Adjunta en modo lectura/escritura.
 - **Retorno:**
 - Un puntero a la memoria compartida en caso de éxito.
 - **(void) -1:* En caso de error, se establece errno.
-

Relación entre las funciones

- **pipe():** Usado para comunicación básica y sencilla entre procesos relacionados.
- **shmget() y shmat():** Son más adecuados para compartir grandes cantidades de datos entre procesos, ya que permiten acceso directo a regiones de memoria compartida.

Estos mecanismos son básicos pero esenciales para la **Comunicación Interprocesos (IPC)** en Linux.

2. Capture, compile y ejecute el siguiente programa. Observe su funcionamiento.

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#define VALOR 1
int main(void)
{
    int desc_arch[2];
    char bufer[100];
    if(pipe(desc_arch) != 0)
        exit(1);
    if(fork() == 0)
    {
        while(VALOR)
        {
            read(desc_arch[0], bufer, sizeof(bufer));
            printf("Se recibió: %s\n", bufer);
        }
    }
    else{
        while(VALOR)
        {
            printf("Escriba un mensaje: ");
            fgets(bufer, sizeof(bufer), stdin);
            write(desc_arch[1], bufer, strlen(bufer)+1);
        }
    }
    return 0;
}
```

```
said@said-VirtualBox:~/Descargas$ ./p1
Escriba un mensaje: linux
Escriba un mensaje: Se recibió: linux

linux
Escriba un mensaje: Se recibió: linux

linux
Escriba un mensaje: Se recibió: linux
```

3. A través del sitio MSDN, investigue el funcionamiento de las llamadas al sistema: `CreatePipe()`, `GetStartupInfo()`, `GetStdHandle()`, `WaitForSingleObject()`. Explique los argumentos y retorno de la función.

CreatePipe

La función `CreatePipe` crea un *pipe* anónimo que permite la comunicación entre dos procesos. Este tipo de *pipe* es unidireccional, lo que significa que un extremo se usa para escribir y el otro para leer.

- **Argumentos:**

1. **hReadPipe:** Un puntero que recibe un identificador del extremo de lectura del *pipe*.
2. **hWritePipe:** Un puntero que recibe un identificador del extremo de escritura del *pipe*.
3. **lpPipeAttributes:** Una estructura `SECURITY_ATTRIBUTES` que define si los identificadores pueden heredarse. Si `binheritHandle` es `TRUE`, los procesos hijos pueden heredar los identificadores.
4. **nSize:** Tamaño del búfer del *pipe* en bytes. Si se especifica 0, se usa el tamaño por defecto del sistema.

- **Retorno:** Devuelve `TRUE` si se ejecuta correctamente, o `FALSE` si ocurre un error. Si falla, se puede usar `GetLastError` para obtener más detalles del error.
-

GetStartupInfo

La función `GetStartupInfo` llena una estructura `STARTUPINFO` con información sobre cómo debe inicializarse una aplicación.

- **Argumentos:**

1. **lpStartupInfo:** Un puntero a una estructura `STARTUPINFO` que se llena con información como ventanas, configuración de consola, y *handles* estándar.

- **Retorno:** No devuelve un valor específico, ya que solo rellena la estructura proporcionada
-

GetStdHandle

La función GetStdHandle obtiene un identificador para un dispositivo de entrada o salida estándar (como consola).

- **Argumentos:**

1. **nStdHandle:** Especifica qué identificador se solicita. Puede ser `STD_INPUT_HANDLE`, `STD_OUTPUT_HANDLE`, o `STD_ERROR_HANDLE`.

- **Retorno:** Devuelve el identificador del dispositivo estándar solicitado. Si falla, devuelve NULL y se puede usar GetLastError para obtener información del error.
-

WaitForSingleObject

La función WaitForSingleObject detiene la ejecución de un hilo hasta que el objeto especificado esté en un estado señalado o hasta que expire un tiempo límite.

- **Argumentos:**

1. **hHandle:** El identificador del objeto a esperar (como un hilo o evento).
2. **dwMilliseconds:** Tiempo máximo (en milisegundos) para esperar el estado señalado. Puede ser INFINITE para esperar indefinidamente.

- **Retorno:** Devuelve un código que indica el estado del objeto o el tiempo de espera. Algunos códigos comunes son `WAIT_OBJECT_0` (el objeto está en estado señalado) y `WAIT_TIMEOUT` (se alcanzó el tiempo límite sin cambios)

4. Capture, compile y ejecute los siguientes programas. Observe su funcionamiento. Ejecute de la siguiente forma:
C:\>nombre_programa_padre nombre_programa_hijo

```
#include <windows.h>
#include <stdio.h>
#include <string.h>
int main (int argc, char *argv[])
{
    char mensaje[]="Tuberias en Windows";
    DWORD escritos;
    HANDLE hLecturaPipe, hEscrituraPipe;
    PROCESS_INFORMATION piHijo;
    STARTUPINFO siHijo;
    SECURITY_ATTRIBUTES pipeSeg =
    {sizeof(SECURITY_ATTRIBUTES), NULL, TRUE};
    /* Obtención de información para la inicialización del proceso hijo */
    GetStartupInfo (&siHijo);
    /* Creación de la tubería sin nombre */
    CreatePipe (&hLecturaPipe, &hEscrituraPipe, &pipeSeg, 0);
    /* Escritura en la tubería sin nombre */
    WriteFile(hEscrituraPipe, mensaje, strlen(mensaje)+1, &escritos, NULL);

    siHijo.hStdInput = hLecturaPipe;
    siHijo.hStdError = GetStdHandle (STD_ERROR_HANDLE);
    siHijo.hStdOutput = GetStdHandle (STD_OUTPUT_HANDLE);
    siHijo.dwFlags = STARTF_USESTDHANDLES;
    CreateProcess (NULL, argv[1], NULL, NULL,
    TRUE, /* Hereda el proceso hijo los manejadores de la tubería del padre */
    0, NULL, NULL, &siHijo, &piHijo);
    WaitForSingleObject (piHijo.hProcess, INFINITE);
    printf("Mensaje recibido en el proceso hijo, termina el proceso padre\n");
    CloseHandle(hLecturaPipe);
    CloseHandle(hEscrituraPipe);
    CloseHandle(piHijo.hThread);
    CloseHandle(piHijo.hProcess);
    return 0;
}
```

```
/* Programa hijo.c */
#include <windows.h>
#include <stdio.h>
int main ()
{
    char mensaje[20];
    DWORD leidos;
    HANDLE hStdIn = GetStdHandle(STD_INPUT_HANDLE);

    /* Lectura desde la tubería sin nombre */
    ReadFile(hStdIn, mensaje, sizeof(mensaje), &leidos, NULL);
    printf("Mensaje recibido del proceso padre: %s\n", mensaje);
    CloseHandle(hStdIn);
    printf("Termina el proceso hijo, continua el proceso padre\n");
    return 0;
}
```

```
C:\Users\vbcs1>gcc p1w.c -o padre  
C:\Users\vbcs1>gcc p2w.c -o hijo  
C:\Users\vbcs1>padre hijo  
Mensaje recibido del proceso padre: Tuberias en Windows  
Termina el proceso hijo, continua el proceso padre  
Mensaje recibido en el proceso hijo, termina el proceso padre
```

5. Programe una aplicación que cree un proceso hijo a partir de un proceso padre, el proceso padre enviará al proceso hijo, a través de una tubería, dos matrices de 10 x 10 a multiplicar por parte del hijo, mientras tanto el proceso hijo creará un hijo de él, al cual enviará dos matrices de 10 x 10 a sumar en el proceso hijo creado, nuevamente el envío de estos valores será a través de una tubería. Una vez calculado el resultado de la suma, el proceso hijo del hijo devolverá la matriz resultante a su abuelo (vía tubería). A su vez, el proceso hijo devolverá la matriz resultante de la multiplicación que realizó a su padre. Finalmente, el proceso padre obtendrá la matriz inversa de cada una de las matrices recibidas y el resultado lo guardará en un archivo para cada matriz inversa obtenida. Programe esta aplicación tanto para Linux como para Windows utilizando las tuberías de cada sistema operativo.

LINUX:

```

#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <time.h>
#define N 10
void llenar(int matriz[N][N]){
    int i, j;
    for (i = 0; i < N; i++){
        for (j = 0; j < N; j++){
            matriz[i][j] = rand() % 101;
        }
    }
}
void guardarMatriz(double matriz[N][N], int archivo) {
    char buffer[256];
    for (int i = 0; i < N; i++) {
        int len = 0;
        for (int j = 0; j < N; j++) {
            len += sprintf(buffer + len, "%lf\t", matriz[i][j]);
        }
        buffer[len++] = '\n';
        write(archivo, buffer, len);
    }
    write(archivo, "\n", 1);
}
void impMatriz(int matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++)
            printf("%d\t", matriz[i][j]);
        printf("\n");
    }
}
void impInversa(double matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++)
            printf("%lf ", matriz[i][j]);
        printf("\n");
    }
}

```

```

void multiplicacion(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++){
                resultado[i][j] += matrizuno[i][k] * matrizdos[k][j];
            }
        }
    }
}

void suma(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = matrizuno[i][j] + matrizdos[i][j];
        }
    }
}

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++){
        for (int col = 0; col < n; col++){
            if (fila != p && col != q){
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1){
                    j = 0;
                    i++;
                }
            }
        }
    }
}

```

```

int DeterminanteMatriz(int matriz[N][N], int n){
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];
    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++){
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1){
        adjunta[0][0] = 1;
        return;
    }
    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

void inversa(int matriz[N][N], double inversa[N][N]){
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }
    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

```

```
int main(){
    srand(time(NULL));
    int tub1[2];
    int tub2[2];
    int matrizA[N][N];
    int matrizB[N][N];
    if (pipe(tub1) != 0 || pipe(tub2) != 0){
        exit(1);
    }
    if (fork() == 0){
        read(tub1[0], matrizA, sizeof(matrizA));
        read(tub1[0], matrizB, sizeof(matrizB));
        int resultado[N][N];
        multiplicacion(matrizA, matrizB, resultado);
        write(tub1[1], resultado, sizeof(resultado));
        write(tub2[1], matrizA, sizeof(matrizA));
        write(tub2[1], matrizB, sizeof(matrizB));
        if (fork() == 0) {
            read(tub2[0], matrizA, sizeof(matrizA));
            read(tub2[0], matrizB, sizeof(matrizB));
            int resultado[N][N];
            suma(matrizA, matrizB, resultado);
            write(tub2[1], resultado, sizeof(resultado));
            exit(0);
        }
        else{
            wait(NULL);
        }
    }
}
```

```

else{
    llenar(matrizA);
    llenar(matrizB);
    printf("Matriz A:\n");
    impMatriz(matrizA);
    printf("\nMatriz B:\n");
    impMatriz(matrizB);
    write(tub1[1], matrizA, sizeof(matrizA));
    write(tub1[1], matrizB, sizeof(matrizB));
    wait(NULL);
    int resMul[N][N];
    int resSum[N][N];
    printf("\nMultiplicacion:\n");
    read(tub1[0], resMul, sizeof(resMul));
    impMatriz(resMul);
    read(tub2[0], resSum, sizeof(resSum));
    printf("\nSuma:\n");
    impMatriz(resSum);
    double invMul[N][N];
    double invSum[N][N];
    inversa(resMul, invMul);
    inversa(resSum, invSum);
    printf("\nInversa de la multiplicacion\n");
    impInversa(invMul);
    printf("\nInversa de la suma\n");
    impInversa(invSum);
    int archivo = open("inversaMul.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (archivo < 0) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    write(archivo, "Inversa de la multiplicacion:\n", 30);
    guardarMatriz(invMul, archivo);
    close(archivo);
    int archivo2 = open("inversaSum.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (archivo < 0) {
        perror("Error al abrir el archivo");
        exit(1);
    }
}

```

```

    write(archivo, "Inversa de la suma:\n", 21);
    guardarMatriz(invSum, archivo);
    close(archivo);
    exit(0);
}

return 0;
}

```

```
said@said-VirtualBox:~/Descargas$ ./tub
```

```
Matriz A:
```

94	67	88	93	15	77	20	22	32	96
49	5	97	87	22	44	79	67	83	32
16	53	40	9	86	7	20	25	34	90
63	27	22	51	19	38	27	5	60	59
68	8	30	64	62	18	74	40	51	23
72	68	76	78	43	27	86	63	52	19
18	81	47	40	31	32	78	58	38	3
83	72	12	12	35	40	98	76	80	48
99	17	15	40	96	58	67	81	87	85
66	4	65	12	11	97	11	55	20	15

```
Matriz B:
```

59	3	87	37	82	21	77	79	63	56
27	61	39	8	67	0	33	33	81	19
17	13	91	83	25	68	45	2	22	65
17	81	68	3	17	16	92	60	95	54
82	88	82	21	63	48	21	96	82	69
81	65	82	71	47	73	38	58	76	27
23	93	74	57	63	92	73	54	17	34
7	100	88	89	20	50	3	7	11	85
76	93	49	23	29	63	97	68	20	38
61	9	97	34	67	59	92	39	12	9

```
Multiplicacion:
```

26801	27271	46963	24477	29602	26407	38668	28453	31576	25023
22068	35610	43526	27214	22579	31020	35404	25741	23163	28371
19536	20885	29413	13873	20539	18381	20794	19372	16995	16019
19138	19517	27779	12993	18869	17316	26501	20874	18353	14716
19629	28724	33596	17575	21184	22160	27309	25173	21118	22191
21945	36514	43175	24921	27123	27422	33572	27820	28127	28598
15133	30269	29917	18842	19592	20685	21551	18740	20263	19818
25153	36035	40603	24076	29449	28039	32224	28493	23053	24203
33710	40174	50583	27217	33145	33291	38152	36808	28814	30426
17143	18050	29574	21515	16139	19514	17448	15802	16698	17967

```
Suma:
```

153	70	175	130	97	98	97	101	95	152
76	66	136	95	89	44	112	100	164	51
33	66	131	92	111	75	65	27	56	155
80	108	90	54	36	54	119	65	155	113
150	96	112	85	125	66	95	136	133	92
153	133	158	149	90	100	124	121	128	46
41	174	121	97	94	124	151	112	55	37
90	172	100	101	55	90	101	83	91	133
175	110	64	63	125	121	164	149	107	123
127	13	162	46	78	156	103	94	32	24


```

Inversa de la multiplicacion
-0.196312 -1.200581 0.593818 2.481500 0.905698 1.917872 -1.322185 -0.8352
28 -2.063213 -1.729142
-2.339169 0.548592 1.023236 -0.455243 -1.541001 0.252276 1.569175 -0.9728
07 -0.520467 -0.113507
2.663963 -1.200423 0.206156 0.486229 -1.137733 -0.452564 -0.822280 -1.761
805 1.180044 -2.117270
-0.987368 2.006924 0.116152 0.874666 -0.265208 -1.319135 1.918911 -1.2340
33 -0.481011 -0.028159
-1.742234 -2.321557 -1.648815 -1.356868 -0.076068 -2.176003 -1.119010 -0.
892950 -1.093965 0.457516
1.216425 1.027833 -1.298588 1.064099 0.616433 0.176514 0.145328 0.562415
2.658825 -0.718059
2.166773 -0.302590 -1.331191 -0.707640 -1.326568 -1.359305 2.645364 -1.96
2875 0.673960 -1.613354
-2.529471 -0.375708 1.389943 0.857721 2.410535 -0.516270 -0.751662 0.6448
41 -0.943679 0.344902
-0.422882 -2.413279 -1.642312 -1.908487 2.608076 1.594815 0.924113 -0.942
759 1.835978 0.762723
0.303752 0.879920 -1.665977 2.124230 -2.361732 1.193989 -0.155018 2.08981
4 -2.628286 0.272228

```

```

Inversa de la suma
-0.306743 0.908669 -0.797946 0.425035 0.837608 -0.102002 0.247989 0.52377
5 0.697864 0.493015
-0.912041 0.231262 0.376871 0.878327 0.319376 0.397110 -0.067234 1.022710
0.017053 0.429150
-0.654740 1.012481 -0.259170 0.717496 -0.993502 -0.801608 -0.188410 -0.95
4041 -0.368226 -0.968310
-0.517403 0.676617 0.135328 0.337585 -0.946940 1.019191 0.978867 -0.43407
4 -0.617832 -0.203681
-0.993983 -0.727370 -0.350600 -0.904586 -0.917822 -0.104874 0.529633 0.22
3745 0.176652 0.710179
0.779384 -0.003010 -1.014735 -0.734563 0.106595 0.589021 0.449113 0.95549
4 0.009346 -0.260144
-0.061138 0.882038 -0.794920 0.953140 0.371123 -0.494836 -0.837367 0.6204
20 -0.336270 -0.023341
-0.863363 0.162561 -0.486167 -0.024815 0.920383 0.911733 0.943892 -0.3275
48 -0.370863 0.731921
0.900157 0.609886 -0.794412 -0.189654 -0.456939 -0.984348 0.492268 0.8854
05 -0.336843 -1.017068
-0.784975 -0.303905 -0.111749 0.906047 0.931233 0.159444 -0.675022 -0.748
145 -0.410650 0.040445

```



inversaSum.
txt



inversaMul.t
xt

Inversa de la multiplicacion:

-0.196312	-1.200581	0.593818	2.481500	
0.905698				
1.917872	-1.322185	-0.835228	-2.063213	-1.
729142				
-2.339169	0.548592			
1.023236	-0.455243	-1.541001	0.252276	
1.569175	-0.972807	-0.520467	-0.113507	
2.663963	-1.200423	0.206156		
0.486229	-1.137733	-0.452564	-0.822280	-1.
761805	1.180044	-2.117270		
-0.987368	2.006924	0.116152		
0.874666	-0.265208	-1.319135		
1.918911	-1.234033	-0.481011	-0.028159	
-1.742234	-2.321557	-1.648815	-1.356868	-0.

Inversa de la suma:

\00-0.306743	0.908669	-0.797946	0.425035	
0.837608	-0.102002	0.247989	0.523775	
0.697864	0.493015			
-0.912041	0.231262	0.376871	0.878327	
0.319376	0.397110	-0.067234	1.022710	
0.017053	0.429150			
-0.654740	1.012481	-0.259170		
0.717496	-0.993502	-0.801608	-0.188410	-0.
954041	-0.368226	-0.968310		
-0.517403	0.676617	0.135328		
0.337585	-0.946940	1.019191		
0.978867	-0.434074	-0.617832	-0.203681	
-0.993983	-0.727370	-0.350600	-0.904586	-0.

WINDOWS

Padre.c

```
#include <windows.h>
#include <stdio.h>
#include <string.h>
#include <time.h>
#define N 10
void llenar(int matriz[N][N]){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = rand() % 101;
        }
    }
}
void impInversaArchivo(double matriz[N][N], FILE *archivo){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            fprintf(archivo, "%lf\t", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fprintf(archivo, "\n");
}
void imprimir(int matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }
}
void impInversa(double matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++)
            printf("%lf ", matriz[i][j]);
        printf("\n");
    }
}
```

```

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++){
        for (int col = 0; col < n; col++){
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int DeterminanteMatriz(int matriz[N][N], int n){
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];
    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++){
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1){
        adjunta[0][0] = 1;
        return;
    }
    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

```

```

void inversa(int matriz[N][N], double inversa[N][N]){
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0){
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }
    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

int main(int argc, char *argv[]) {
    srand(time(NULL));
    int MatrizA[N][N], MatrizB[N][N];
    llenar(MatrizA);
    llenar(MatrizB);
    printf("Matriz A:\n");
    imprimir(MatrizA);
    printf("\nMatriz B:\n");
    imprimir(MatrizB);
    HANDLE lectura1, escritura1;
    HANDLE lectura0, escritura0;
    SECURITY_ATTRIBUTES pipeSeg = {sizeof(SECURITY_ATTRIBUTES), NULL, TRUE};
    DWORD bytesEscritos, bytesLeidos;
    if (!CreatePipe(&lectura1, &escritura1, &pipeSeg, 0)) {
        printf("Error creando pipe para entrada.\n");
        return 1;
    }
    if (!CreatePipe(&lectura0, &escritura0, &pipeSeg, 0)) {
        printf("Error creando pipe para salida.\n");
        return 1;
    }
    STARTUPINFO sHijo = {0};
    PROCESS_INFORMATION pHijo = {0};
    sHijo.cb = sizeof(STARTUPINFO);
    sHijo.hStdInput = lectura1;
    sHijo.hStdOutput = escritura0;
    sHijo.hStdError = GetStdHandle(STD_ERROR_HANDLE);
    sHijo.dwFlags = STARTF_USESTDHANDLES;

```

```

if (!WriteFile(escritura1, MatrizA, sizeof(MatrizA), &bytesEscritos, NULL)) {
    printf("Error escribiendo matriz A.\n");
    return 1;
}
if (!WriteFile(escritura1, MatrizB, sizeof(MatrizB), &bytesEscritos, NULL)) {
    printf("Error escribiendo matriz B.\n");
    return 1;
}
if (!CreateProcess(NULL, argv[1], NULL, NULL, TRUE, 0, NULL, NULL, &sHijo, &pHijo)) {
    printf("Error creando proceso hijo.\n");
    return 1;
}
WaitForSingleObject(pHijo.hProcess, INFINITE);
int resultado[N][N];
double inversa1[N][N];
FILE *archivo;
if (!ReadFile(lectura0, resultado, sizeof(resultado), &bytesLeidos, NULL)) {
    printf("Error leyendo resultado de la multiplicación.\n");
    return 1;
}
inversa(resultado, inversa1);
printf("\nInversa de la Multiplicacion:\n");
impInversa(inversa1);
archivo = fopen("inversaMul.txt", "w");
if (!archivo) {
    printf("Error abriendo archivo para multiplicación.\n");
    return 1;
}
impInversaArchivo(inversa1, archivo);
fclose(archivo);
if (!ReadFile(lectura0, resultado, sizeof(resultado), &bytesLeidos, NULL)) {
    printf("Error leyendo resultado de la suma.\n");
    return 1;
}
inversa(resultado, inversa1);
printf("Inversa de la Suma:\n");
impInversa(inversa1);
archivo = fopen("inversaSum.txt", "w");
if (!archivo) {

```

```

    if (!archivo) {
        printf("Error abriendo archivo para suma.\n");
        return 1;
    }
    impInversaArchivo(inversa1, archivo);
    fclose(archivo);
    CloseHandle(lectura1);
    CloseHandle(escritura1);
    CloseHandle(lectura0);
    CloseHandle(escritura0);
    CloseHandle(pHijo.hThread);
    CloseHandle(pHijo.hProcess);
    return 0;
}

```

Hijo.c

```

#include <windows.h>
#include <stdio.h>
#include <string.h>
#define N 10
void multiplicacion(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++){
                resultado[i][j] += matrizuno[i][k] * matrizdos[k][j];
            }
        }
    }
}

```

```

int main(){
    HANDLE lecturaPipe = GetStdHandle(STD_INPUT_HANDLE);
    DWORD lectura;
    HANDLE escrituraPipe = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD escritura;
    HANDLE lectura1, escritura1;
    HANDLE lectura0, escritura0;
    SECURITY_ATTRIBUTES pipeSeg = { sizeof(SECURITY_ATTRIBUTES), NULL, TRUE };
    STARTUPINFO sHijo;
    PROCESS_INFORMATION pHijo;
    CreatePipe(&lectura1, &escritura1, &pipeSeg, 0);
    CreatePipe(&lectura0, &escritura0, &pipeSeg, 0);
    int MatrizA[N][N];
    int MatrizB[N][N];
    ReadFile(lecturaPipe, MatrizA, sizeof(MatrizA), &lectura, NULL);
    ReadFile(lecturaPipe, MatrizB, sizeof(MatrizB), &lectura, NULL);
    int Resultado[N][N];
    multiplicacion(MatrizA, MatrizB, Resultado);
    WriteFile(escrituraPipe, Resultado, sizeof(Resultado), &escritura, NULL);
    GetStartupInfo(&sHijo);
    WriteFile(escritura1, MatrizA, sizeof(MatrizA), &escritura, NULL);
    WriteFile(escritura1, MatrizB, sizeof(MatrizB), &escritura, NULL);
    sHijo.hStdInput = lectura1;
    sHijo.hStdError = GetStdHandle(STD_ERROR_HANDLE);
    sHijo.hStdOutput = escritura0;
    sHijo.dwFlags = STARTF_USESTDHANDLES;
    CreateProcess(NULL, "nieto", NULL, NULL, TRUE, 0, NULL, NULL, &sHijo, &pHijo);
    WaitForSingleObject(pHijo.hProcess, INFINITE);
    ReadFile(lectura0, Resultado, sizeof(Resultado), &lectura, NULL);
    WriteFile(escrituraPipe, Resultado, sizeof(Resultado), &escritura, NULL);
    CloseHandle(lecturaPipe);
    CloseHandle(escrituraPipe);
    CloseHandle(lectura1);
    CloseHandle(lectura0);
    CloseHandle(escritura1);
    CloseHandle(escritura0);
    return 0;
}

```


Nieto.c

```
#include <windows.h>
#include <stdio.h>
#define N 10
void sumar(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = matrizuno[i][j] + matrizdos[i][j];
        }
    }
}
int main(){
    HANDLE lectura1 = GetStdHandle(STD_INPUT_HANDLE);
    DWORD lectura;
    HANDLE escritura1 = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD escritura;
    int MatrizA[N][N];
    int MatrizB[N][N];
    ReadFile(lectura1, MatrizA, sizeof(MatrizA), &lectura, NULL);
    ReadFile(lectura1, MatrizB, sizeof(MatrizB), &lectura, NULL);
    int Resultado[N][N];
    sumar(MatrizA, MatrizB, Resultado);
    WriteFile(escritura1, Resultado, sizeof(Resultado), &escritura, NULL);
    CloseHandle(lectura1);
    CloseHandle(escritura1);
    return 0;
}
```

```
C:\Users\vbcs1>gcc padre.c -o padre
```

```
C:\Users\vbcs1>gcc hijo.c -o hijo
```

```
C:\Users\vbcs1>gcc nieto.c -o nieto
```

```
C:\Users\vbcs1>padre hijo
```

Matriz A:

52	2	49	10	79	12	67	8	48	11
50	21	36	10	47	73	37	57	87	22
10	52	3	6	33	96	23	58	74	54
79	39	12	58	6	67	80	43	21	97
69	5	64	52	4	29	63	50	57	51
64	96	35	36	52	80	52	25	96	46
92	19	95	11	56	62	76	84	87	64
44	34	77	32	63	52	60	78	43	14
29	63	17	98	48	6	83	12	46	12
51	4	93	67	83	10	41	4	21	85

Matriz B:

48	24	97	18	2	41	90	65	36	55
18	75	69	36	59	5	90	62	73	22
93	31	71	95	69	33	55	70	35	10
1	53	86	42	82	26	42	94	60	93
36	82	22	61	56	45	94	5	3	96
21	14	78	76	23	59	70	39	35	87
0	74	78	85	21	73	55	98	73	45
10	53	56	4	13	9	31	36	26	3
79	13	12	35	83	16	84	43	58	14
73	79	33	25	46	45	23	97	0	37

Inversa de la Multiplicacion:

```
0.348557 0.069764 0.174717 -0.629968 -0.632966 1.344967 0.672675 -0.630714 1.485322 -1.292217
-0.341315 0.499533 -1.508212 -1.122005 0.234657 -0.707894 0.256950 -0.382859 -1.625937 1.251190
-0.428466 -0.801138 -1.609773 -1.264944 0.932749 -0.900622 -0.372403 1.300577 -1.108092 1.383811
0.708356 -0.868937 -0.380791 1.100959 -1.267366 0.642579 -1.480680 1.236244 -1.191714 1.274424
-0.513497 -1.239574 -0.837150 -0.776584 -0.206402 -0.998054 0.969018 -1.136265 1.104602 -0.197144
-1.437780 0.720569 -1.198922 -1.000651 1.488255 1.015003 -1.562541 1.416074 -0.492068 0.708731
0.280458 1.494956 -1.427200 -1.008795 0.100522 1.644467 -1.370447 0.906206 -0.527114 -0.572382
1.564206 0.757549 0.568055 -0.590082 -0.950965 -1.188469 0.098630 1.360179 0.496433 0.183794
-1.084171 -1.645753 0.684312 0.797903 -0.248904 1.017359 0.485648 -0.363901 -0.903187 1.539200
-0.131900 1.106039 0.592219 -1.056881 -0.048576 -0.514826 0.225922 1.372931 -1.002686 -1.326890
```

Inversa de la Suma:

```
-0.604376 -0.605260 -0.076759 -0.227885 -0.752793 1.078045 -0.641193 -0.340208 -0.782451 0.556796
-0.633323 0.737590 0.462816 -0.958195 -0.514739 -0.125160 -0.863738 -0.710533 0.709498 0.831583
-0.033702 0.702268 0.159255 -0.133732 0.381060 -0.381768 -0.281167 1.087603 -0.082977 0.737600
0.258609 0.510011 -0.921299 0.533351 -0.079086 0.819318 -0.273483 0.879312 1.032863 0.587476
-0.241825 0.200532 -0.370856 0.515707 -0.690838 -0.926157 0.464336 0.311375 0.532351 -0.605792
0.455038 -1.091651 -0.077051 -0.430902 0.197207 -0.398616 -0.775714 -0.822987 -0.471454 -1.037861
0.207117 -0.014323 -0.949678 0.271517 -0.116666 0.486712 0.275773 1.059991 -0.441837 -0.261060
0.028998 -0.003251 0.479118 0.318706 -0.544749 0.630693 -0.408121 0.703245 -0.406217 0.370197
-0.852821 0.641949 -0.208985 -0.412877 0.502341 1.011872 -0.325704 0.695903 -0.444728 -0.252929
0.139270 0.546187 0.986736 1.019805 -0.306148 0.630814 -0.955511 1.123294 -0.178409 -0.056600
```



inversaMul

30/11/2024 15:17



inversaSum

30/11/2024 15:17

0.348557	0.069764	0.174717	-0.629968	-0.632966	1.344967	0.672675
-0.630714	1.485322	-1.292217				
-0.341315	0.499533	-1.508212	-1.122005	0.234657	-0.707894	0.256950
-0.382859	-1.625937	1.251190				
-0.428466	-0.801138	-1.609773	-1.264944	0.932749	-0.900622	-0.372403
1.300577	-1.108092	1.383811				
0.708356	-0.868937	-0.380791	1.100959	-1.267366	0.642579	-1.480680
1.236244	-1.191714	1.274424				
-0.513497	-1.239574	-0.837150	-0.776584	-0.206402	-0.998054	0.969018
-1.136265	1.104602	-0.197144				
-1.437780	0.720569	-1.198922	-1.000651	1.488255	1.015003	-1.562541
1.416074	-0.492068	0.708731				
0.280458	1.494956	-1.427200	-1.008795	0.100522	1.644467	-1.370447
0.906206	-0.527114	-0.572382				
1.564206	0.757549	0.568055	-0.590082	-0.950965	-1.188469	0.098630
1.360179	0.496433	0.183794				
-1.084171	-1.645753	0.684312	0.797903	-0.248904	1.017359	0.485648
-0.363901	-0.903187	1.539200				
-0.131900	1.106039	0.592219	-1.056881	-0.048576	-0.514826	0.225922
1.372931	-1.002686	-1.326890				
-0.604376	-0.605260	-0.076759	-0.227885	-0.752793	1.078045	-0.641193
-0.340208	-0.782451	0.556796				
-0.633323	0.737590	0.462816	-0.958195	-0.514739	-0.125160	-0.863738
-0.710533	0.709498	0.831583				
-0.033702	0.702268	0.159255	-0.133732	0.381060	-0.381768	-0.281167
1.087603	-0.082977	0.737600				
0.258609	0.510011	-0.921299	0.533351	-0.079086	0.819318	-0.273483
0.879312	1.032863	0.587476				
-0.241825	0.200532	-0.370856	0.515707	-0.690838	-0.926157	0.464336
0.311375	0.532351	-0.605792				
0.455038	-1.091651	-0.077051	-0.430902	0.197207	-0.398616	-0.775714
-0.822987	-0.471454	-1.037861				
0.207117	-0.014323	-0.949678	0.271517	-0.116666	0.486712	0.275773
1.059991	-0.441837	-0.261060				
0.028998	-0.003251	0.479118	0.318706	-0.544749	0.630693	-0.408121
0.703245	-0.406217	0.370197				
-0.852821	0.641949	-0.208985	-0.412877	0.502341	1.011872	-0.325704
0.695903	-0.444728	-0.252929				
0.139270	0.546187	0.986736	1.019805	-0.306148	0.630814	-0.955511
1.123294	-0.178409	-0.056600				

6. Capture, compile y ejecute los siguientes programas para Linux. Observe su funcionamiento.

```
#include <sys/types.h> /* Cliente de la memoria compartida */
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>
#define TAM_MEM 27 /* Tamaño de la memoria compartida en bytes */
int main(){
    int shmid;
    key_t llave;
    char *shm, *s;
    llave = 5678;
    if ((shmid = shmget(llave, TAM_MEM, 0666)) < 0){
        perror("Error al obtener memoria compartida: shmget");
        exit(-1);
    }
    if ((shm = shmat(shmid, NULL, 0)) == (char *) -1) {
        perror("Error al enlazar la memoria compartida: shmat");
        exit(-1);
    }
    for (s = shm; *s != '\0'; s++) {
        putchar(*s);
    }
    putchar('\n');
    *shm = '*';
    exit(0);
}
```

```
#include <sys/types.h> /* Servidor de la memoria compartida */
#include <sys/ipc.h> /* (ejecutar el servidor antes de ejecutar el cliente) */
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define TAM_MEM 27 /* Tamaño de la memoria compartida en bytes */
int main() {
    char c;
    int shmid;
    key_t llave;
    char *shm, *s;
    llave = 5678;
    if ((shmid = shmget(llave, TAM_MEM, IPC_CREAT | 0666)) < 0) {
        perror("Error al obtener memoria compartida: shmget");
        exit(-1);
    }
    if ((shm = shmat(shmid, NULL, 0)) == (char *) -1) {
        perror("Error al enlazar la memoria compartida: shmat");
        exit(-1);
    }
    s = shm;
    for (c = 'a'; c <= 'z'; c++) {
        *s++ = c;
    }
    *s = '\0';
    while (*shm != '*') {
        sleep(1);
    }
    exit(0);
}
```

```
said@said-VirtualBox:~/Descargas$ ./com1
abcdefghijklmnopqrstuvwxyz
```

7. Capture, compile y ejecute los siguientes programas para Windows. Observe su funcionamiento.

```
#include <windows.h> /* Cliente de la memoria compartida */
#include <stdio.h>
#define TAM_MEM 27 /* Tamaño de la memoria compartida en bytes */
int main(void)
{
    HANDLE hMemCom;
    char *idMemCompartida = "MemoriaCompartida";
    char *apDatos, *apTrabajo;
    // Abre el mapeo de archivo de la memoria compartida
    if ((hMemCom = OpenFileMapping(
        FILE_MAP_ALL_ACCESS, // Acceso lectura/escritura de la memoria compartida
        FALSE,               // No se hereda el nombre
        idMemCompartida      // Identificador de la memoria compartida
    )) == NULL) {
        printf("No se accedió a la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    // Mapea la vista del archivo a la memoria del proceso
    if ((apDatos = (char *)MapViewOfFile(
        hMemCom,              // Manejador del mapeo
        FILE_MAP_ALL_ACCESS, // Permiso de lectura/escritura en la memoria
        0,
        0,
        TAM_MEM
    )) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    // Lee datos de la memoria compartida y los imprime
    for (apTrabajo = apDatos; *apTrabajo != '\0'; apTrabajo++) {
        putchar(*apTrabajo);
    }
    putchar('\n');
    *apDatos = '*';
    UnmapViewOfFile(apDatos);
    CloseHandle(hMemCom);
    return 0;
}
```

```

#include <windows.h> /* Servidor de la memoria compartida */
#include <stdio.h> /* (ejecutar el servidor antes de ejecutar el cliente)*/
#define TAM_MEM 27 /* Tamaño de la memoria compartida en bytes */
int main(void) {
    HANDLE hMemCom;
    char *idMemCompartida = "MemoriaCompartida";
    char *apDatos, *apTrabajo, c;
    if ((hMemCom = CreateFileMapping(
        INVALID_HANDLE_VALUE, // Usa memoria compartida
        NULL, // Seguridad por defecto
        PAGE_READWRITE, // Acceso lectura/escritura a la memoria
        0, // Tamaño máximo parte alta de un DWORD
        TAM_MEM, // Tamaño máximo parte baja de un DWORD
        idMemCompartida // Identificador de la memoria compartida
    )) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((apDatos = (char *)MapViewOfFile(
        hMemCom, // Manejador del mapeo
        FILE_MAP_ALL_ACCESS, // Permiso de lectura/escritura en la memoria
        0,
        0,
        TAM_MEM
    )) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    apTrabajo = apDatos;
    for (c = 'a'; c <= 'z'; c++) {
        *apTrabajo++ = c;
    }
    *apTrabajo = '\0';
    while (*apDatos != '*') {
        Sleep(1);
    }
    UnmapViewOfFile(apDatos);
    CloseHandle(hMemCom);
    return 0;
}

```

```

C:\Users\vbcs1>com
abcdefghijklmnopqrstuvwxyz

```

8. Programe nuevamente la aplicación del punto 5 utilizando en esta ocasión memoria compartida en lugar de tuberías (utilice tantas memorias compartidas como requiera). Programe esta aplicación tanto para Linux como para Windows utilizando la memoria compartida de cada sistema operativo.

LINUX:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <fcntl.h>
#define TAMANIO 10
void llenarMatriz(int matriz[TAMANIO][TAMANIO]){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            matriz[fila][columna] = rand() % 101;
        }
    }
}
void guardarMatriz(double matriz[TAMANIO][TAMANIO], int archivo) {
    char buffer[256];
    for (int i = 0; i < TAMANIO; i++) {
        int len = 0;
        for (int j = 0; j < TAMANIO; j++) {
            len += sprintf(buffer + len, "%lf\t", matriz[i][j]);
        }
        buffer[len++] = '\n';
        write(archivo, buffer, len);
    }
    write(archivo, "\n", 1);
}
void imprimirMatrizEnteros(int matriz[TAMANIO][TAMANIO]){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            printf("%d\t", matriz[fila][columna]);
        }
        printf("\n");
    }
}
```

```

void imprimirMatrizReales(double matriz[TAMANIO][TAMANIO]){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            printf("%lf ", matriz[fila][columna]);
            printf("\n");
        }
    }
}

void multiplicarMatrices(int matrizA[TAMANIO][TAMANIO], int matrizB[TAMANIO][TAMANIO], int resultado[TAMANIO][TAMANIO]){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            resultado[fila][columna] = 0;
            for (int k = 0; k < TAMANIO; k++){
                resultado[fila][columna] += matrizA[fila][k] * matrizB[k][columna];
            }
        }
    }
}

void sumarMatrices(int matrizA[TAMANIO][TAMANIO], int matrizB[TAMANIO][TAMANIO], int resultado[TAMANIO][TAMANIO]){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            resultado[fila][columna] = matrizA[fila][columna] + matrizB[fila][columna];
        }
    }
}

void calcularCofactor(int matriz[TAMANIO][TAMANIO], int temporal[TAMANIO][TAMANIO], int filaOmitida, int columnaOmitida, int tamActual){
    int filaTemp = 0, columnaTemp = 0;
    for (int fila = 0; fila < tamActual; fila++){
        for (int columna = 0; columna < tamActual; columna++){
            if (fila != filaOmitida && columna != columnaOmitida){
                temporal[filaTemp][columnaTemp++] = matriz[fila][columna];
                if (columnaTemp == tamActual - 1){
                    columnaTemp = 0;
                    filaTemp++;
                }
            }
        }
    }
}

```

```

int calcularDeterminante(int matriz[TAMANIO][TAMANIO], int tamActual){
    int determinante = 0;
    if (tamActual == 1)
        return matriz[0][0];
    int cofactor[TAMANIO][TAMANIO];
    int signo = 1;
    for (int columna = 0; columna < tamActual; columna++){
        calcularCofactor(matriz, cofactor, 0, columna, tamActual);
        determinante += signo * matriz[0][columna] * calcularDeterminante(cofactor, tamActual - 1);
        signo = -signo;
    }
    return determinante;
}

void calcularAdjunta(int matriz[TAMANIO][TAMANIO], double adjunta[TAMANIO][TAMANIO], int tamActual){
    if (tamActual == 1){
        adjunta[0][0] = 1;
        return;
    }
    int cofactor[TAMANIO][TAMANIO];
    int signo;
    for (int fila = 0; fila < tamActual; fila++){
        for (int columna = 0; columna < tamActual; columna++){
            calcularCofactor(matriz, cofactor, fila, columna, tamActual);
            signo = ((fila + columna) % 2 == 0) ? 1 : -1;
            adjunta[columna][fila] = signo * calcularDeterminante(cofactor, tamActual - 1);
        }
    }
}

void calcularInversa(int matriz[TAMANIO][TAMANIO], double inversa[TAMANIO][TAMANIO]){
    double determinante = (double)calcularDeterminante(matriz, TAMANIO);
    if (determinante == 0){
        printf("La matriz es singular, no tiene inversa.\n");
        return;
    }
    double adjunta[TAMANIO][TAMANIO];
    calcularAdjunta(matriz, adjunta, TAMANIO);
    for (int fila = 0; fila < TAMANIO; fila++)
        for (int columna = 0; columna < TAMANIO; columna++)
            inversa[fila][columna] = adjunta[fila][columna] / determinante;
}

```

```

void escribirMemoria(int (*matriz)[TAMANIO], int *shm){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            *shm = matriz[fila][columna];
            shm++;
        }
    }
}

void leerMemoria(int (*matriz)[TAMANIO], int *shm){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            matriz[fila][columna] = *shm;
            shm++;
        }
    }
}

int main(){
    srand(time(NULL));
    int matrizA[TAMANIO][TAMANIO], matrizB[TAMANIO][TAMANIO];
    int matrizMultiplicacion[TAMANIO][TAMANIO], matrizSuma[TAMANIO][TAMANIO];
    key_t llaveA = 1000, llaveB = 1001, llaveMulti = 1002, llaveSuma = 1003;
    int shmIdA = shmget(llaveA, sizeof(int[TAMANIO][TAMANIO]), IPC_CREAT | 0666);
    int shmIdB = shmget(llaveB, sizeof(int[TAMANIO][TAMANIO]), IPC_CREAT | 0666);
    int shmIdMulti = shmget(llaveMulti, sizeof(int[TAMANIO][TAMANIO]), IPC_CREAT | 0666);
    int shmIdSuma = shmget(llaveSuma, sizeof(int[TAMANIO][TAMANIO]), IPC_CREAT | 0666);
    int(*shmA)[TAMANIO] = shmat(shmIdA, NULL, 0);
    int(*shmB)[TAMANIO] = shmat(shmIdB, NULL, 0);
    int(*shmMulti)[TAMANIO] = shmat(shmIdMulti, NULL, 0);
    int(*shmSuma)[TAMANIO] = shmat(shmIdSuma, NULL, 0);
    if (fork() == 0){
        leerMemoria(matrizA, (int *)shmA);
        leerMemoria(matrizB, (int *)shmB);
        multiplicarMatrices(matrizA, matrizB, matrizMultiplicacion);
        escribirMemoria(matrizMultiplicacion, (int *)shmMulti);
        escribirMemoria(matrizA, (int *)shmA);
        escribirMemoria(matrizB, (int *)shmB);
    }
}

```



```

    if (fork() == 0){
        leerMemoria(matrizA, (int *)shmA);
        leerMemoria(matrizB, (int *)shdB);
        sumarMatrices(matrizA, matrizB, matrizSuma);
        escribirMemoria(matrizSuma, (int *)shmSuma);
        exit(0);
    }
    wait(NULL);
    exit(0);
}
else{
    llenarMatriz(matrizA);
    llenarMatriz(matrizB);
    printf("Matriz A:\n");
    imprimirMatrizEnteros(matrizA);
    printf("\nMatriz B:\n");
    imprimirMatrizEnteros(matrizB);
    if (shmIdA < 0) {
        perror("Error al obtener memoria compartida para la primera matriz: shmget");
        exit(1);
    }
    if (shmIdB < 0) {
        perror("Error al obtener memoria compartida para la segunda matriz: shmget");
        exit(1);
    }
    if (shmIdMulti < 0) {
        perror("Error al obtener memoria compartida para el resultado de la multiplicación: shmget");
        exit(1);
    }
    if (shmIdSuma < 0) {
        perror("Error al obtener memoria compartida para el resultado de la suma: shmget");
        exit(1);
    }
    if (shmA == (void *)-1) {
        perror("Error al enlazar la memoria compartida para la primera matriz: shmat");
        exit(1);
    }
}

```

```

if (shmB == (void *)-1) {
    perror("Error al enlazar la memoria compartida para la segunda matriz: shmat");
    exit(1);
}
if (shmMulti == (void *)-1) {
    perror("Error al enlazar la memoria compartida para el resultado de la multiplicación: shmat");
    exit(1);
}
if (shmSuma == (void *)-1) {
    perror("Error al enlazar la memoria compartida para el resultado de la suma: shmat");
    exit(1);
}
escribirMemoria(matrizA, (int *)shmA);
escribirMemoria(matrizB, (int *)shmB);
wait(NULL);
leerMemoria(matrizMultiplicacion, (int *)shmMulti);
leerMemoria(matrizSuma, (int *)shmSuma);
printf("\nResultado de la multiplicación:\n");
imprimirMatrizEnteros(matrizMultiplicacion);
printf("\nResultado de la suma:\n");
imprimirMatrizEnteros(matrizSuma);
double inversa_multi[TAMANIO][TAMANIO];
double inversa_sum[TAMANIO][TAMANIO];
calcularInversa(matrizMultiplicacion, inversa_multi);
calcularInversa(matrizSuma, inversa_sum);
printf("\nInversa de la multiplicación:\n");
imprimirMatrizReales(inversa_multi);
printf("\nInversa de la suma:\n");
imprimirMatrizReales(inversa_sum);
int archivo = open("inversaMul.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
if (archivo < 0) {
    perror("Error al abrir el archivo");
    exit(1);
}
write(archivo, "Inversa de la multiplicacion:\n", 30);
guardarMatriz(inversa_multi, archivo);
close(archivo);

```

```

    int archivo2 = open("inversaSum.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (archivo2 < 0) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    write(archivo2, "Inversa de la suma:\n", 21);
    guardarMatriz(inversa_sum, archivo2);
    close(archivo2);
}
return 0;
}

```

Matriz A:									
53	38	7	81	78	7	77	67	43	4
4	3	90	4	27	78	91	31	47	87
44	84	18	46	62	15	23	51	69	98
17	89	1	91	69	45	64	45	11	7
15	16	77	5	87	4	83	77	1	96
64	45	80	48	92	7	29	81	59	98
44	42	86	45	32	55	56	96	66	68
69	82	50	46	87	2	16	35	80	17
31	9	29	77	23	87	84	52	67	8
15	10	50	68	21	48	22	77	10	54

Matriz B:									
10	79	35	26	91	88	29	6	23	75
24	20	50	19	97	73	5	46	91	72
21	5	48	37	73	69	86	61	45	62
82	56	40	16	82	98	4	10	3	94
85	94	13	34	12	76	73	17	88	63
55	8	35	2	46	74	71	98	35	16
59	83	38	65	99	19	62	69	97	32
28	81	25	7	82	38	83	20	21	37
84	77	45	85	79	57	58	16	20	59
32	79	41	70	10	6	55	38	75	51

Resultado de la multiplicación:									
22295	25208	28384	18081	24907	24245	21764	25132	19635	29633
28805	33655	32265	29113	33556	42129	27357	37541	30414	50034
18485	20908	23171	14135	19636	24894	16056	25715	22536	30467
28571	34551	25262	23198	31216	28641	30345	28460	25379	40363
17314	25868	22267	17826	24022	18978	21050	25953	19951	30536
26331	33147	23318	26433	30698	31599	27814	29693	23745	44109
25283	28183	22166	23813	25460	25171	25211	24992	24118	33433
23187	25751	22933	22476	26064	27108	23968	25630	21967	33906
24072	28835	22753	19833	26390	27155	24086	24866	19309	35463
29401	34693	23972	26158	34690	34579	30831	27594	19290	43354

Resultado de la suma:									
63	117	42	107	169	95	106	73	66	79
28	23	140	23	124	151	96	77	138	159
65	89	66	83	135	84	109	112	114	160
99	145	41	107	151	143	68	55	14	101
100	110	90	39	99	80	156	94	89	159
119	53	115	50	138	81	100	179	94	114
103	125	124	110	131	74	118	165	163	100
97	163	75	53	169	40	99	55	101	54
115	86	74	162	102	144	142	68	87	67
47	89	91	138	31	54	77	115	85	105

Inversa de la multiplicación:

```
-0.097720 -1.347413 0.065737 -1.609511 -0.799122 -0.603193 -1.565944 -0.3
40212 1.245027 0.719871
-0.565830 -0.046757 -1.354217 -0.654703 -0.363688 -1.535931 1.615158 0.13
3126 0.062947 0.005763
0.088375 1.662213 -0.792529 -0.642717 -0.131301 1.066040 0.853417 -0.1752
26 -0.712963 0.128640
0.136740 0.275554 -0.705910 1.431864 -1.472516 -1.167372 -0.704017 -0.568
146 -1.380685 1.744917
1.261127 -0.217263 0.792375 0.457631 1.234208 -1.560096 1.020312 -1.28855
9 0.490875 -1.214570
0.782239 -1.155129 -0.808209 1.028507 0.081832 -1.778929 0.468841 -1.0702
71 1.492704 0.582270
0.858801 -1.128846 1.422960 -0.832190 -0.480632 -1.675350 1.465612 0.7395
12 -1.559379 0.500779
-0.778539 1.552826 -1.163677 0.172296 1.436230 -1.608679 0.985063 1.04234
4 -1.457857 1.539530
0.912513 0.409637 -1.254879 -0.620669 1.663448 -0.274315 0.194195 1.42447
6 0.879034 -1.677461
-0.703392 -1.080739 -0.877711 -1.295591 -1.422807 -1.335553 -0.043129 0.9
43905 1.001312 0.435630
```

Inversa de la suma:

```
0.791474 -0.211934 -0.310962 -0.684923 -0.358586 -0.948241 -0.499936 -0.2
84424 -0.238097 -0.246578
-0.255396 -0.171739 0.932600 -0.061393 -0.748184 -0.452631 0.727180 -1.02
1895 -0.642130 -0.688212
-0.484310 0.134848 0.031154 0.440659 0.757612 -0.492304 -0.885845 0.96143
2 0.289631 0.946208
0.214811 0.499581 -0.654155 0.507002 -0.828686 -0.557785 0.471880 0.52770
8 -0.827078 0.937333
0.313566 0.136105 -0.538315 -0.813085 0.820354 0.130861 -0.252366 -0.0809
39 -0.445145 0.522673
0.345882 0.975840 -1.040279 0.033845 -0.345906 0.313072 0.804857 0.918689
-0.451421 0.131265
-0.846047 -1.007399 -0.747885 -0.068382 -0.718522 0.159514 0.854097 0.382
943 0.635229 -0.223732
-0.384949 0.540156 -0.685969 -0.226722 -1.045803 -0.885061 -0.101384 -0.6
54928 -0.117526 -0.345845
-0.298822 -0.734909 0.635730 -0.379096 -0.295283 -0.100851 0.426692 -1.03
6157 -0.738309 0.302567
-0.953477 0.690442 0.897939 0.379174 0.204670 0.555052 -0.762895 0.193053
-0.818794 0.934750
```



inversaSum.
txt



inversaMul.t
xt

Inversa de la multiplicacion:

-0.097720	-1.347413			
0.065737	-1.609511	-0.799122	-0.603193	-1.
565944	-0.340212	1.245027	0.719871	
-0.565830	-0.046757	-1.354217	-0.654703	-0.
363688	-1.535931	1.615158	0.133126	0.062947
0.005763				
0.088375				
1.662213	-0.792529	-0.642717	-0.131301	
1.066040	0.853417	-0.175226	-0.712963	
0.128640				
0.136740	0.275554	-0.705910		
1.431864	-1.472516	-1.167372	-0.704017	-0.
568146	-1.380685	1.744917		

Inversa de la suma:

\000.791474	-0.211934	-0.310962	-0.684923	-0.
358586	-0.948241	-0.499936	-0.284424	-0.238097
-0.246578				
-0.255396	-0.171739			
0.932600	-0.061393	-0.748184	-0.452631	
0.727180	-1.021895	-0.642130	-0.688212	
0.484310	0.134848	0.031154	0.440659	
0.757612	-0.492304	-0.885845	0.961432	
0.289631	0.946208			
0.214811	0.499581	-0.654155		
0.507002	-0.828686	-0.557785	0.471880	
0.527708	-0.827078	0.937333		
0.313566	0.136105	-0.538315	-0.813085	

WINDOWS

Padre.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))
void LlenarMatriz(int matriz[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = rand() % 101;
        }
    }
}
void ImprimirMatriz(int matriz[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }
}
void impInversaArchivo(double matriz[N][N], FILE *archivo){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            fprintf(archivo, "%lf\t", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fprintf(archivo, "\n");
}
void ImprimirInversa(double matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++)
            printf("%lf ", matriz[i][j]);
        printf("\n");
    }
}
```

```

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++){
        for (int col = 0; col < n; col++){
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1){
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int DeterminanteMatriz(int matriz[N][N], int n){
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];
    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++){
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1){
        adjunta[0][0] = 1;
        return;
    }
    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

```

```

void InversaMatriz(int matriz[N][N], double inversa[N][N]){
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0){
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }
    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

HANDLE CrearMemoriaCompartida(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(
        INVALID_HANDLE_VALUE,
        NULL,
        PAGE_READWRITE,
        0,
        TAM_MEM,
        nombre
    )) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(
        hMemoria,
        FILE_MAP_ALL_ACCESS,
        0,
        0,
        TAM_MEM
    )) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}

```



```

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void EsperarMemoriaCompartida(const char* nombre, HANDLE* hMemCom){
    while ((*hMemCom = OpenFileMapping(
        FILE_MAP_ALL_ACCESS,
        FALSE,
        nombre
    )) == NULL) {
        Sleep(100);
    }
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

int* MapearMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((apDatos = (int *)MapViewOfFile(
        hMemCom,
        FILE_MAP_ALL_ACCESS,
        0,
        0,
        TAM_MEM
    )) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

```

```

int main(int argc, char* argv[]){
    srand(time(NULL));
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    LlenarMatriz(Matrizuno);
    LlenarMatriz(Matrizdos);
    printf("Matriz A:\n");
    ImprimirMatriz(Matrizuno);
    printf("\nMatriz B:\n");
    ImprimirMatriz(Matrizdos);
    int* apDatosUno;
    int* apDatosDos;
    HANDLE hMemComUno = CrearMemoriaCompartida("Memoria_Compartida_MU_PH", &apDatosUno);
    HANDLE hMemComDos = CrearMemoriaCompartida("Memoria_Compartida_MD_PH", &apDatosDos);
    EscribirMemoriaCompartida(Matrizuno, apDatosUno);
    EscribirMemoriaCompartida(Matrizdos, apDatosDos);
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    if (!CreateProcess(NULL, argv[1], NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)){
        printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
        return 1;
    }
    while (apDatosUno[0] != -1 && apDatosDos[0] != -1){
        Sleep(1);
    }
    UnmapViewOfFile(apDatosUno);
    UnmapViewOfFile(apDatosDos);
    CloseHandle(hMemComUno);
    CloseHandle(hMemComDos);
    HANDLE hMemComResmulti;
    int* apDatosResmulti;
    EsperarMemoriaCompartida("Memoria_Compartida_Res_Multi", &hMemComResmulti);
    apDatosResmulti = MapearMemoriaCompartida("Memoria_Compartida_Res_Multi", hMemComResmulti);
    int Resultado_multi[N][N];
    LeerMemoriaCompartida(Resultado_multi, apDatosResmulti);
    apDatosResmulti[0] = -1;
    UnmapViewOfFile(apDatosResmulti);
    CloseHandle(hMemComResmulti);
}

```

```

HANDLE hMemComResum;
int* apDatosResum;
EsperarMemoriaCompartida("Memoria_Compartida_Res_Sum", &hMemComResum);
apDatosResum = MapearMemoriaCompartida("Memoria_Compartida_Res_Sum", hMemComResum);
int Resultado_sum[N][N];
LeerMemoriaCompartida(Resultado_sum, apDatosResum);
apDatosResum[0] = -1;
UnmapViewOfFile(apDatosResum);
CloseHandle(hMemComResum);
double InversaMul[N][N], InversaS[N][N];
printf("\n\nInversa de la multiplicacion:\n");
InversaMatriz(Resultado_multi, InversaMul);
ImprimirInversa(InversaMul);
printf("\n\nInversa de la suma:\n");
InversaMatriz(Resultado_sum, InversaS);
ImprimirInversa(InversaS);
FILE *archivo;
archivo = fopen("inversaMul.txt", "w");
if (!archivo) {
    printf("Error abriendo archivo para multiplicación.\n");
    return 1;
}
impInversaArchivo(InversaMul, archivo);
fclose(archivo);
archivo = fopen("inversaSum.txt", "w");
if (!archivo) {
    printf("Error abriendo archivo para suma.\n");
    return 1;
}
impInversaArchivo(InversaS, archivo);
fclose(archivo);
WaitForSingleObject(pi.hProcess, INFINITE);
CloseHandle(pi.hProcess);
CloseHandle(pi.hThread);
return 0;
}

```

Hijo.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))
void MultiplicarMatrices(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++){
                resultado[i][j] += matrizuno[i][k] * matrizdos[k][j];
            }
        }
    }
}

HANDLE CrearMemoriaCompartida(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(
        INVALID_HANDLE_VALUE,
        NULL,
        PAGE_READWRITE,
        0,
        TAM_MEM,
        nombre
    )) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(
        hMemoria,
        FILE_MAP_ALL_ACCESS,
        0,
        0,
        TAM_MEM
    )) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}
```

```

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void ErrorMemoriaCompartida(const char* nombre, HANDLE* hMemCom){
    if ((*hMemCom = OpenFileMapping(
        FILE_MAP_ALL_ACCESS,
        FALSE,
        nombre
    )) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
}

int* MapearMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((apDatos = (int *)MapViewOfFile(
        hMemCom,
        FILE_MAP_ALL_ACCESS,
        0,
        0,
        TAM_MEM
    )) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

```

```

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE hMemComMatrizUno;
    int* apDatosMatrizUno;
    ErrorMemoriaCompartida("Memoria_Compartida_MU_PH", &hMemComMatrizUno);
    apDatosMatrizUno = MapearMemoriaCompartida("Memoria_Compartida_MU_PH", hMemComMatrizUno);
    LeerMemoriaCompartida(Matrizuno, apDatosMatrizUno);
    apDatosMatrizUno[0] = -1;
    UnmapViewOfFile(apDatosMatrizUno);
    CloseHandle(hMemComMatrizUno);
    HANDLE hMemComMatrizDos;
    int* apDatosMatrizDos;
    ErrorMemoriaCompartida("Memoria_Compartida_MD_PH", &hMemComMatrizDos);
    apDatosMatrizDos = MapearMemoriaCompartida("Memoria_Compartida_MD_PH", hMemComMatrizDos);
    LeerMemoriaCompartida(Matrizdos, apDatosMatrizDos);
    apDatosMatrizDos[0] = -1;
    UnmapViewOfFile(apDatosMatrizDos);
    CloseHandle(hMemComMatrizDos);
    int Resultado[N][N];
    MultiplicarMatrices(Matrizuno, Matrizdos, Resultado);
    int* apDatosResmulti;
    HANDLE hMemComResmulti = CrearMemoriaCompartida("Memoria_Compartida_Res_Multi", &apDatosResmulti);
    EscribirMemoriaCompartida(Resultado, apDatosResmulti);
    while (apDatosResmulti[0] != -1) {
        Sleep(1);
    }
    UnmapViewOfFile(apDatosResmulti);
    CloseHandle(hMemComResmulti);
    int* apDatosUno;
    int* apDatosDos;
    HANDLE hMemComUno = CrearMemoriaCompartida("Memoria_Compartida_MU_HN", &apDatosUno);
    HANDLE hMemComDos = CrearMemoriaCompartida("Memoria_Compartida_MD_HN", &apDatosDos);
    EscribirMemoriaCompartida(Matrizuno, apDatosUno);
    EscribirMemoriaCompartida(Matrizdos, apDatosDos);
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

```

```

    if (!CreateProcess(NULL, "nieto", NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)){
        printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
        return 1;
    }
    while (apDatosUno[0] != -1 && apDatosDos[0] != -1){
        Sleep(1);
    }
    UnmapViewOfFile(apDatosUno);
    UnmapViewOfFile(apDatosDos);
    CloseHandle(hMemComUno);
    CloseHandle(hMemComDos);
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    return 0;
}

```

Nieto.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))
void SumarMatrices(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = matrizuno[i][j] + matrizdos[i][j];
        }
    }
}
HANDLE CrearMemoriaCompartida(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(
        INVALID_HANDLE_VALUE,
        NULL,
        PAGE_READWRITE,
        0,
        TAM_MEM,
        nombre
    )) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(
        hMemoria,
        FILE_MAP_ALL_ACCESS,
        0,
        0,
        TAM_MEM
    )) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}
```

```

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void ErrorMemoriaCompartida(const char* nombre, HANDLE* hMemCom){
    if ((*hMemCom = OpenFileMapping(
        FILE_MAP_ALL_ACCESS,
        FALSE,
        nombre
    )) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
}

int* MapearMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((apDatos = (int *)MapViewOfFile(
        hMemCom,
        FILE_MAP_ALL_ACCESS,
        0,
        0,
        TAM_MEM
    )) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

```



```

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE hMemComMatrizUno;
    int* apDatosMatrizUno;
    ErrorMemoriaCompartida("Memoria_Compartida_MU_HN", &hMemComMatrizUno);
    apDatosMatrizUno = MapearMemoriaCompartida("Memoria_Compartida_MU_HN", hMemComMatrizUno);
    LeerMemoriaCompartida(Matrizuno, apDatosMatrizUno);
    apDatosMatrizUno[0] = -1;
    UnmapViewOfFile(apDatosMatrizUno);
    CloseHandle(hMemComMatrizUno);
    HANDLE hMemComMatrizDos;
    int* apDatosMatrizDos;
    ErrorMemoriaCompartida("Memoria_Compartida_MD_HN", &hMemComMatrizDos);
    apDatosMatrizDos = MapearMemoriaCompartida("Memoria_Compartida_MD_HN", hMemComMatrizDos);
    LeerMemoriaCompartida(Matrizdos, apDatosMatrizDos);
    apDatosMatrizDos[0] = -1;
    UnmapViewOfFile(apDatosMatrizDos);
    CloseHandle(hMemComMatrizDos);
    int Resultado[N][N];
    SumarMatrices(Matrizuno, Matrizdos, Resultado);
    int* apDatosResum;
    HANDLE hMemComResum = CrearMemoriaCompartida("Memoria_Compartida_Res_Sum", &apDatosResum);
    EscribirMemoriaCompartida(Resultado, apDatosResum);
    while (apDatosResum[0] != -1) {
        Sleep(1);
    }
    UnmapViewOfFile(apDatosResum);
    CloseHandle(hMemComResum);
    return 0;
}

```

C:\Users\vbcs1>gcc padre.c -o padre

C:\Users\vbcs1>gcc hijo.c -o hijo

C:\Users\vbcs1>gcc nieto.c -o nieto

C:\Users\vbcs1>padre hijo

Matriz A:

75	80	36	2	83	15	96	60	18	85
39	57	16	89	31	0	86	63	26	0
95	93	22	83	79	16	56	40	28	83
16	70	96	90	51	55	12	50	71	68
100	10	76	77	95	4	35	67	67	80
47	14	20	72	93	6	33	71	91	17
24	48	8	66	55	10	86	23	61	32
59	15	9	16	12	4	17	46	91	40
23	58	45	31	85	29	22	81	41	48
6	92	55	31	83	13	37	98	81	25

Matriz B:

56	5	9	80	76	39	1	45	62	19
85	20	39	57	88	91	55	64	98	83
39	22	5	50	85	76	98	68	98	40
42	48	59	50	62	38	29	31	11	8
69	86	3	40	41	19	24	71	46	68
24	23	77	11	31	4	2	43	27	98
63	53	60	91	69	58	15	62	82	48
88	50	100	13	40	96	85	20	26	49
95	14	96	33	3	49	92	22	58	36
37	65	3	98	36	23	32	69	28	56

Inversa de la multiplicacion:

-1.707219	-0.747121	0.231675	-0.838588	-1.865872	-0.958085	-1.283213	-1.379207	-1.868514	-1.845512
-1.291053	-1.631736	-1.305115	1.417634	-0.279051	-2.014750	1.983048	-0.079369	1.237633	-1.032267
0.746387	-0.945840	1.556442	-0.273157	-1.676311	0.322585	0.558260	0.782640	0.782396	1.558033
1.009377	2.035510	1.119295	1.235963	-0.814898	0.693940	-0.211024	1.504747	0.890678	-1.401265
1.087646	-1.071925	-0.340706	0.223268	-1.242490	1.630999	-0.952613	1.896772	1.705861	2.068009
1.414484	-1.781848	1.025893	-1.660148	2.034726	1.792805	-0.889532	-1.707528	1.175173	-1.069061
1.847132	-1.409423	-1.735277	1.243036	0.851364	-2.139600	1.065050	1.730691	-1.446626	1.122573
-0.529296	1.229592	1.248687	-0.737203	-0.580924	1.784975	0.167653	1.472955	1.616613	-0.546975
-0.094950	-2.055565	-0.052811	0.728876	1.335667	2.026553	-2.130939	2.042058	-1.611245	0.863596
-0.968963	-0.263588	-0.481444	1.973425	-0.163603	-1.998410	1.919715	-0.025317	-1.200478	-0.097704

Inversa de la suma:

-0.383707	0.466511	-0.348671	1.136487	1.144448	0.354549	0.829391	-0.877293	-1.116361	0.192405
0.266097	-0.869469	0.705833	1.079310	-1.011158	1.116630	-1.128627	0.906745	-0.144826	1.030442
-0.364530	-1.045441	-0.952280	0.161898	-0.159315	0.906241	-1.042677	-0.315729	0.790148	0.529900
0.815355	0.489107	0.477235	-0.684386	0.058288	0.016741	0.607778	0.759538	0.524204	1.093499
-0.594360	0.722501	0.210787	-1.071655	-0.942937	0.351782	-0.685116	0.269539	-0.095946	-1.008058
-0.147397	-0.387585	0.656949	0.476108	-0.981541	-0.750829	0.391254	-0.484042	-0.629803	-1.100097
-0.453844	-0.299829	0.371908	0.785715	-0.518967	-0.723460	-0.259009	0.650497	-0.954560	-0.449256
0.666614	0.341156	-0.405245	-0.712449	0.159851	-0.688349	1.070095	0.019986	0.999404	-0.786442
-0.938430	-0.787549	-0.978719	-0.220374	0.530876	-0.459301	-1.082595	-0.136278	-1.058834	-0.598473
-0.051030	-0.678888	-1.072776	0.737626	-0.289467	-0.783793	-0.148538	0.099368	0.155872	0.090926



inversaM
ul



inversaS
um

-1.707219	-0.747121	0.231675	-0.838588	-1.865872	-0.958085	-1.283213
-1.379207	-1.868514	-1.845512				
-1.291053	-1.631736	-1.305115	1.417634	-0.279051	-2.014750	1.983048
-0.079369	1.237633	-1.032267				
0.746387	-0.945840	1.556442	-0.273157	-1.676311	0.322585	0.558260
0.782640	0.782396	1.558033				
1.009377	2.035510	1.119295	1.235963	-0.814898	0.693940	-0.211024
1.504747	0.890678	-1.401265				
1.087646	-1.071925	-0.340706	0.223268	-1.242490	1.630999	-0.952613
1.896772	1.705861	2.068009				
1.414484	-1.781848	1.025893	-1.660148	2.034726	1.792805	-0.889532
-1.707528	1.175173	-1.069061				
1.847132	-1.409423	-1.735277	1.243036	0.851364	-2.139600	1.065050
1.730691	-1.446626	1.122573				
-0.529296	1.229592	1.248687	-0.737203	-0.580924	1.784975	0.167653
1.472955	1.616613	-0.546975				
-0.094950	-2.055565	-0.052811	0.728876	1.335667	2.026553	-2.130939
2.042058	-1.611245	0.863596				
-0.968963	-0.263588	-0.481444	1.973425	-0.163603	-1.998410	1.919715
-0.025317	-1.200478	-0.097704				
-0.383707	0.466511	-0.348671	1.136487	1.144448	0.354549	0.829391
-0.877293	-1.116361	0.192405				
0.266097	-0.869469	0.705833	1.079310	-1.011158	1.116630	-1.128627
0.906745	-0.144826	1.030442				
-0.364530	-1.045441	-0.952280	0.161898	-0.159315	0.906241	-1.042677
-0.315729	0.790148	0.529900				
0.815355	0.489107	0.477235	-0.684386	0.058288	0.016741	0.607778
0.759538	0.524204	1.093499				
-0.594360	0.722501	0.210787	-1.071655	-0.942937	0.351782	-0.685116
0.269539	-0.095946	-1.008058				
-0.147397	-0.387585	0.656949	0.476108	-0.981541	-0.750829	0.391254
-0.484042	-0.629803	-1.100097				
-0.453844	-0.299829	0.371908	0.785715	-0.518967	-0.723460	-0.259009
0.650497	-0.954560	-0.449256				
0.666614	0.341156	-0.405245	-0.712449	0.159851	-0.688349	1.070095
0.019986	0.999404	-0.786442				
-0.938430	-0.787549	-0.978719	-0.220374	0.530876	-0.459301	-1.082595
-0.136278	-1.058834	-0.598473				
-0.051030	-0.678888	-1.072776	0.737626	-0.289467	-0.783793	-0.148538
0.099368	0.155872	0.090926				

9. Programe la misma aplicación del punto 5 de la práctica 5 en su sección de Linux, pero en esta ocasión cada proceso que realiza una operación de matrices, comunicará con un IPC el resultado de la operación al proceso que recolectará los resultados. Realice la versión del programa usando tuberías (en Linux y Windows), y la versión usando memoria compartida (en Linux y Windows).

LINUX TUBERIAS

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <sys/wait.h>
#define N 10

void imprimirMatriz(double A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%.2lf\t", A[i][j]);
        }
        printf("\n");
    }
}

void imprimirMatrizI(int A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%d\t", A[i][j]);
        }
        printf("\n");
    }
}

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

void sumarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] + B[i][j];
        }
    }
}
```

```

void restarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] - B[i][j];
        }
    }
}

void multiplicarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++) {
                resultado[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void transponerMatriz(int matriz[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[j][i] = matriz[i][j];
        }
    }
}

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++) {
        for (int col = 0; col < n; col++) {
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

```

```

int DeterminanteMatriz(int matriz[N][N], int n) {
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];

    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++) {
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1) {
        adjunta[0][0] = 1;
        return;
    }

    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

```

```

void InversaMatriz(int matriz[N][N], double inversa[N][N]) {
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }

    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

int main(){
    srand(time(NULL));
    int tubSum[2];
    int tubRes[2];
    int tubMul[2];
    int tubTras[2];
    int tubInv[2];
    if (pipe(tubSum) != 0 || pipe(tubRes) != 0 || pipe(tubMul) != 0 || pipe(tubTras) != 0 || pipe(tubInv) != 0){
        exit(1);
    }
    int A[N][N], B[N][N];
    double resultado[N][N];
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 101;
            B[i][j] = rand() % 101;
        }
    }
    printf("\nMatriz A:\n");
    imprimirMatrizI(A);
    printf("\nMatriz B:\n");
    imprimirMatrizI(B);
}

```

```

pid_t pid;
for (int i = 1; i <= 6; i++) {
    pid = fork();
    if (pid == 0) {
        switch (i) {
            case 1:
                sumarMatrices(A, B, resultado);
                escribirMatriz("suma.txt", resultado);
                write(tubSum[1], resultado, sizeof(resultado));
                break;
            case 2:
                restarMatrices(A, B, resultado);
                escribirMatriz("resta.txt", resultado);
                write(tubRes[1], resultado, sizeof(resultado));
                break;
            case 3:
                multiplicarMatrices(A, B, resultado);
                escribirMatriz("multiplicacion.txt", resultado);
                write(tubMul[1], resultado, sizeof(resultado));
                break;
            case 4:
                transponerMatriz(A, resultado);
                escribirMatriz("transpuesta_A.txt", resultado);
                write(tubTras[1], resultado, sizeof(resultado));
                transponerMatriz(B, resultado);
                escribirMatriz("transpuesta_B.txt", resultado);
                write(tubTras[1], resultado, sizeof(resultado));
                break;
            case 5:
                double Inversauno[N][N];
                double Inversados[N][N];
                InversaMatriz(A, Inversauno);
                InversaMatriz(B, Inversados);
                escribirMatriz("inversa_A.txt", Inversauno);
                escribirMatriz("inversa_B.txt", Inversados);
                write(tubInv[1], Inversauno, sizeof(Inversauno));
                write(tubInv[1], Inversados, sizeof(Inversados));
                break;
        }
    }
}

```

```

        case 6:
            printf("\nResultados de la suma:\n");
            read(tubSum[0], resultado, sizeof(resultado));
            imprimirMatriz(resultado);
            printf("\nResultados de la resta:\n");
            read(tubRes[0], resultado, sizeof(resultado));
            imprimirMatriz(resultado);
            printf("\nResultados de la multiplicación:\n");
            read(tubMul[0], resultado, sizeof(resultado));
            imprimirMatriz(resultado);
            printf("\nTranspuesta de la matriz A:\n");
            read(tubTras[0], resultado, sizeof(resultado));
            imprimirMatriz(resultado);
            printf("\nTranspuesta de la matriz B:\n");
            read(tubTras[0], resultado, sizeof(resultado));
            imprimirMatriz(resultado);
            printf("\nInversa de la matriz A:\n");
            read(tubInv[0], resultado, sizeof(resultado));
            imprimirMatriz(resultado);
            printf("\nInversa de la matriz B:\n");
            read(tubInv[0], resultado, sizeof(resultado));
            imprimirMatriz(resultado);
            break;
    }
    exit(0);
}
else{
    wait(NULL);
}
}
for (int i = 1; i <= 6; i++) {
    wait(NULL);
}
return 0;
}

```


Matriz A:

49	41	40	53	61	5	2	92	72	19
87	99	71	100	6	1	71	15	2	0
78	58	32	78	65	97	8	73	45	27
29	38	3	43	53	2	15	77	42	51
83	90	8	36	67	3	57	62	26	100
13	42	94	9	71	7	73	10	25	33
75	53	9	24	51	47	57	26	39	18
61	28	55	81	16	37	16	55	71	94
1	53	61	41	80	75	18	5	74	19
100	47	12	39	79	72	38	71	35	64

Matriz B:

46	95	94	24	43	62	60	31	49	60
45	56	84	82	25	21	9	10	34	30
73	80	15	71	63	17	95	79	38	51
28	66	19	73	43	97	26	14	20	6
47	57	62	53	63	96	7	27	71	45
68	96	85	35	60	53	97	81	12	92
2	98	81	1	90	65	55	17	35	9
93	33	4	80	13	57	91	92	45	22
40	24	79	100	26	6	69	73	86	90
56	82	68	50	32	80	91	99	11	14

Resultados de la suma:

95.00	136.00	134.00	77.00	104.00	67.00	62.00	123.00	121.00	79.00
132.00	155.00	155.00	182.00	31.00	22.00	80.00	25.00	36.00	30.00
151.00	138.00	47.00	149.00	128.00	114.00	103.00	152.00	83.00	78.00
57.00	104.00	22.00	116.00	96.00	99.00	41.00	91.00	62.00	57.00
130.00	147.00	70.00	89.00	130.00	99.00	64.00	89.00	97.00	145.00
81.00	138.00	179.00	44.00	131.00	60.00	170.00	91.00	37.00	125.00
77.00	151.00	90.00	25.00	141.00	112.00	112.00	43.00	74.00	27.00
154.00	61.00	59.00	161.00	29.00	94.00	107.00	147.00	116.00	116.00
41.00	77.00	140.00	141.00	106.00	81.00	87.00	78.00	160.00	109.00
156.00	129.00	80.00	89.00	111.00	152.00	129.00	170.00	46.00	78.00

Resultados de la resta:

3.00	-54.00	-54.00	29.00	18.00	-57.00	-58.00	61.00	23.00	-41.00
42.00	43.00	-13.00	18.00	-19.00	-20.00	62.00	5.00	-32.00	-30.00
5.00	-22.00	17.00	7.00	2.00	80.00	-87.00	-6.00	7.00	-24.00
1.00	-28.00	-16.00	-30.00	10.00	-95.00	-11.00	63.00	22.00	45.00
36.00	33.00	-54.00	-17.00	4.00	-93.00	50.00	35.00	-45.00	55.00
-55.00	-54.00	9.00	-26.00	11.00	-46.00	-24.00	-71.00	13.00	-59.00
73.00	-45.00	-72.00	23.00	-39.00	-18.00	2.00	9.00	4.00	9.00
-32.00	-5.00	51.00	1.00	3.00	-20.00	-75.00	-37.00	26.00	72.00
-39.00	29.00	-18.00	-59.00	54.00	69.00	-51.00	-68.00	-12.00	-71.00
44.00	-35.00	-56.00	-11.00	47.00	-8.00	-53.00	-28.00	24.00	50.00

Resultados de la multiplicación:

24214.00	24124.00	21374.00	30167.00	15930.00	23167.00
24578.00	23518.00	21377.00	18521.00		
18407.00	34028.00	25885.00	24371.00	22064.00	24491.00
21003.00	13672.00	16097.00	13922.00		
30486.00	37870.00	32772.00	33132.00	23792.00	32656.00
32779.00	29040.00	22081.00	26475.00		
18821.00	20375.00	18545.00	22968.00	12769.00	21708.00
19421.00	19165.00	15637.00	12183.00		
25333.00	36504.00	34294.00	28841.00	22084.00	32650.00
27917.00	25132.00	21065.00	17212.00		
17339.00	27210.00	21500.00	20118.00	21217.00	19451.00
21151.00	18112.00	16460.00	14316.00		
17857.00	28672.00	28276.00	19822.00	19240.00	22492.00
21202.00	17076.00	17201.00	18159.00		
26868.00	34368.00	28104.00	31937.00	20508.00	28780.00
34282.00	30900.00	19406.00	20975.00		
21417.00	27672.00	26191.00	27327.00	20731.00	21263.00
23513.00	22238.00	18997.00	22705.00		
28955.00	39236.00	35766.00	29078.00	24506.00	34421.00
32904.00	29098.00	22446.00	24385.00		

Transpuesta de la matriz A:

49.00	87.00	78.00	29.00	83.00	13.00	75.00	61.00	1.00	100.00
41.00	99.00	58.00	38.00	90.00	42.00	53.00	28.00	53.00	47.00
40.00	71.00	32.00	3.00	8.00	94.00	9.00	55.00	61.00	12.00
53.00	100.00	78.00	43.00	36.00	9.00	24.00	81.00	41.00	39.00
61.00	6.00	65.00	53.00	67.00	71.00	51.00	16.00	80.00	79.00
5.00	1.00	97.00	2.00	3.00	7.00	47.00	37.00	75.00	72.00
2.00	71.00	8.00	15.00	57.00	73.00	57.00	16.00	18.00	38.00
92.00	15.00	73.00	77.00	62.00	10.00	26.00	55.00	5.00	71.00
72.00	2.00	45.00	42.00	26.00	25.00	39.00	71.00	74.00	35.00
19.00	0.00	27.00	51.00	100.00	33.00	18.00	94.00	19.00	64.00

Transpuesta de la matriz B:

46.00	45.00	73.00	28.00	47.00	68.00	2.00	93.00	40.00	56.00
95.00	56.00	80.00	66.00	57.00	96.00	98.00	33.00	24.00	82.00
94.00	84.00	15.00	19.00	62.00	85.00	81.00	4.00	79.00	68.00
24.00	82.00	71.00	73.00	53.00	35.00	1.00	80.00	100.00	50.00
43.00	25.00	63.00	43.00	63.00	60.00	90.00	13.00	26.00	32.00
62.00	21.00	17.00	97.00	96.00	53.00	65.00	57.00	6.00	80.00
60.00	9.00	95.00	26.00	7.00	97.00	55.00	91.00	69.00	91.00
31.00	10.00	79.00	14.00	27.00	81.00	17.00	92.00	73.00	99.00
49.00	34.00	38.00	20.00	71.00	12.00	35.00	45.00	86.00	11.00
60.00	30.00	51.00	6.00	45.00	92.00	9.00	22.00	90.00	14.00

Inversa de la matriz A:

1.04	0.88	-1.02	0.88	0.37	-0.68	0.40	-0.69	-1.22	-0.81
-0.33	0.95	-0.45	-0.99	-1.20	1.22	-1.26	-0.74	-0.48	0.40
-0.43	1.08	-0.55	-0.89	-0.89	0.87	-0.04	0.15	-0.83	-1.04
0.43	-1.14	-0.27	1.11	-1.12	-1.17	0.79	-0.44	1.05	0.31
0.64	1.00	0.58	0.41	0.43	-0.93	-0.26	-0.40	-1.32	-0.08
-0.29	-0.77	0.04	-0.23	-0.85	1.31	-0.17	-0.67	0.91	-0.65
-0.56	1.25	0.98	0.68	0.11	0.34	-0.91	0.32	-0.79	0.23
-0.65	-0.94	-0.99	-0.11	-0.57	-0.17	0.33	0.54	0.61	-1.34
-1.13	-0.91	-1.02	-0.90	-0.03	0.80	1.03	-0.60	-0.08	0.30
-1.12	1.17	1.24	-0.44	0.04	-1.03	0.05	-0.31	-1.24	1.20

Inversa de la matriz B:

-1.76	0.50	-2.61	-0.80	0.73	-1.73	0.96	2.30	2.74	-0.18
-0.48	0.41	-0.09	-0.76	2.45	1.63	2.08	2.21	-1.42	-1.40
-2.43	1.26	2.64	0.94	2.28	-0.52	-1.90	2.59	0.89	-0.90
1.33	2.54	-0.00	1.25	2.35	2.73	1.67	-1.49	-0.16	-0.88
2.47	1.10	2.18	1.40	-2.44	-0.15	-0.44	-0.51	-1.54	1.21
0.42	0.70	1.84	1.71	-0.44	-0.53	-0.34	1.69	-1.41	-1.26
0.04	-0.96	0.89	0.51	-1.40	1.90	-0.41	-2.41	-2.72	1.82
-1.39	-0.96	-0.48	-1.77	-2.51	1.91	2.15	2.60	2.38	0.39
1.09	1.23	1.67	1.32	1.71	-0.77	-0.39	-1.62	-2.36	-2.15
2.78	-1.36	-0.21	-2.07	-1.91	-2.65	-1.64	2.03	2.22	-2.22



inversa_B.txt



inversa_A.txt



transpuesta
_B.txt



transpuesta
_A.txt



suma.txt



multiplicaci
on.txt

LINUX COMPARTIDA:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#define N 10
void imprimirMatriz(double A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%.2lf\t",A[i][j]);
        }
        printf("\n");
    }
}
void imprimirMatrizI(int A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%d\t",A[i][j]);
        }
        printf("\n");
    }
}
void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}
```

```
void sumarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] + B[i][j];
        }
    }
}

void restarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] - B[i][j];
        }
    }
}

void multiplicarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++) {
                resultado[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void transponerMatriz(int matriz[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[j][i] = matriz[i][j];
        }
    }
}
```

```

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++) {
        for (int col = 0; col < n; col++) {
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int DeterminanteMatriz(int matriz[N][N], int n) {
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];

    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++) {
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

```

```

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1) {
        adjunta[0][0] = 1;
        return;
    }

    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

void InversaMatriz(int matriz[N][N], double inversa[N][N]) {
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }

    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

void escribirMemoriaD(double (*matriz)[N], double *shm){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++){
            *shm = matriz[i][j];
            shm++;
        }
    }
}

```

```

void leerMemoriaD(double (*matriz)[N], double *shm) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *shm;
            shm++;
        }
    }
}

int main(){
    srand(time(NULL));
    int A[N][N], B[N][N];
    double resultado[N][N];
    key_t key1 = 1001;
    key_t key2 = 1002;
    key_t key3 = 1003;
    key_t key4 = 1004;
    key_t key5 = 1005;
    key_t key6 = 1006;
    key_t key7 = 1007;
    int shmid1 = shmget(key1, sizeof(int[N][N]), IPC_CREAT | 0666);
    int shmid2 = shmget(key2, sizeof(int[N][N]), IPC_CREAT | 0666);
    int shmid3 = shmget(key3, sizeof(int[N][N]), IPC_CREAT | 0666);
    int shmid4 = shmget(key4, sizeof(int[N][N]), IPC_CREAT | 0666);
    int shmid5 = shmget(key5, sizeof(int[N][N]), IPC_CREAT | 0666);
    int shmid6 = shmget(key6, sizeof(int[N][N]), IPC_CREAT | 0666);
    int shmid7 = shmget(key7, sizeof(int[N][N]), IPC_CREAT | 0666);
    double (*shmSum)[N] = shmat(shmid1, NULL, 0);
    double (*shmRes)[N] = shmat(shmid2, NULL, 0);
    double (*shmMul)[N] = shmat(shmid3, NULL, 0);
    double (*shmTraA)[N] = shmat(shmid4, NULL, 0);
    double (*shmTraB)[N] = shmat(shmid5, NULL, 0);
    double (*shmInvA)[N] = shmat(shmid6, NULL, 0);
    double (*shmInvB)[N] = shmat(shmid7, NULL, 0);
    if (shmid1 < 0) {
        perror("Error al obtener memoria compartida para la matriz resultado de suma: shmget");
        exit(1);
    }
    if (shmid2 < 0) {
        perror("Error al obtener memoria compartida para la matriz resultado de resta: shmget");
        exit(1);
    }
}

```



```

if (shmid3 < 0) {
    perror("Error al obtener memoria compartida para la matriz resultado de multiplicación: shmget");
    exit(1);
}
if (shmid4 < 0) {
    perror("Error al obtener memoria compartida para la matriz resultado de traspuesta uno: shmget");
    exit(1);
}
if (shmid5 < 0) {
    perror("Error al obtener memoria compartida para la matriz resultado de traspuesta dos: shmget");
    exit(1);
}
if (shmid6 < 0) {
    perror("Error al obtener memoria compartida para la matriz resultado de inversa uno: shmget");
    exit(1);
}
if (shmid7 < 0) {
    perror("Error al obtener memoria compartida para la matriz resultado de inversa dos: shmget");
    exit(1);
}
if (shmSum == (void *)-1) {
    perror("Error al enlazar la memoria compartida para la matriz resultado de suma: shmat");
    exit(1);
}
if (shmRes == (void *)-1) {
    perror("Error al enlazar la memoria compartida para la matriz resultado de resta: shmat");
    exit(1);
}
if (shmMul == (void *)-1) {
    perror("Error al enlazar la memoria compartida para la matriz resultado de multiplicación: shmat");
    exit(1);
}
if (shmTraA == (void *)-1) {
    perror("Error al enlazar la memoria compartida para la matriz resultado de traspuesta uno: shmat");
    exit(1);
}
if (shmTraB == (void *)-1) {
    perror("Error al enlazar la memoria compartida para la matriz resultado de traspuesta dos: shmat");
    exit(1);
}
}

```

```

if (shmInvA == (void *)-1) {
    perror("Error al enlazar la memoria compartida para la matriz resultado de inversa uno: shmat");
    exit(1);
}
if (shmInvB == (void *)-1) {
    perror("Error al enlazar la memoria compartida para la matriz resultado de inversa dos: shmat");
    exit(1);
}
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        A[i][j] = rand() % 101;
        B[i][j] = rand() % 101;
    }
}
printf("\nMatriz A:\n");
imprimirMatrizI(A);
printf("\nMatriz B:\n");
imprimirMatrizI(B);
pid_t pid;
for (int i = 1; i <= 6; i++) {
    pid = fork();
    if (pid == 0) {
        switch (i) {
            case 1:
                sumarMatrices(A, B, resultado);
                escribirMatriz("suma.txt", resultado);
                escribirMemoriaD(resultado, (double *)shmSum);
                break;
            case 2:
                restarMatrices(A, B, resultado);
                escribirMatriz("resta.txt", resultado);
                escribirMemoriaD(resultado, (double *)shmRes);
                break;
            case 3:
                multiplicarMatrices(A, B, resultado);
                escribirMatriz("multiplicacion.txt", resultado);
                escribirMemoriaD(resultado, (double *)shmMul);
                break;
        }
    }
}

```

case 4:

```
transponerMatriz(A, resultado);  
escribirMatriz("transpuesta_A.txt", resultado);  
escribirMemoriaD(resultado, (double *)shmTraA);  
transponerMatriz(B, resultado);  
escribirMatriz("transpuesta_B.txt", resultado);  
escribirMemoriaD(resultado, (double *)shmTraB);  
break;
```

case 5:

```
double Inversauno[N][N];  
double Inversados[N][N];  
InversaMatriz(A, Inversauno);  
InversaMatriz(B, Inversados);  
escribirMatriz("inversa_A.txt", Inversauno);  
escribirMatriz("inversa_B.txt", Inversados);  
escribirMemoriaD(Inversauno, (double *)shmInvA);  
escribirMemoriaD(Inversados, (double *)shmInvB);  
break;
```

case 6:

```
printf("\nResultados de la suma:\n");  
leerMemoriaD(resultado, (double *)shmSum);  
imprimirMatriz(resultado);  
printf("\nResultados de la resta:\n");  
leerMemoriaD(resultado, (double *)shmRes);  
imprimirMatriz(resultado);  
printf("\nResultados de la multiplicación:\n");  
leerMemoriaD(resultado, (double *)shmMul);  
imprimirMatriz(resultado);  
printf("\nTranspuesta de la matriz A:\n");  
leerMemoriaD(resultado, (double *)shmTraA);  
imprimirMatriz(resultado);  
printf("\nTranspuesta de la matriz B:\n");  
leerMemoriaD(resultado, (double *)shmTraB);  
imprimirMatriz(resultado);  
printf("\nInversa de la matriz A:\n");  
leerMemoriaD(resultado, (double *)shmInvA);  
imprimirMatriz(resultado);
```

```

        printf("\nInversa de la matriz B:\n");
        leerMemoriaD(resultado, (double *)shmInvB);
        imprimirMatriz(resultado);
        break;
    }
    exit(0);
}
else{
    wait(NULL);
}
}
for (int i = 1; i <= 6; i++) {
    wait(NULL);
}
return 0;
}

```

Matriz A:

15	12	9	20	55	25	86	32	45	1
50	6	15	19	92	53	15	18	69	32
86	61	68	45	61	10	3	43	86	90
12	15	6	51	37	99	40	61	85	20
51	55	79	6	26	73	38	98	54	86
32	46	76	12	45	38	28	83	22	2
51	52	96	99	65	5	35	0	91	3
78	64	66	79	68	2	22	73	70	32
88	63	22	66	9	81	42	59	89	61
52	74	84	24	50	20	26	26	79	94

Matriz B:

66	79	4	75	82	37	11	74	2	65
91	0	3	38	84	73	65	92	38	23
57	72	59	80	13	77	83	86	62	43
42	94	33	76	84	90	25	75	72	11
30	21	98	30	96	4	54	44	49	57
51	38	73	26	32	62	59	6	89	48
75	89	16	49	85	63	18	77	80	1
7	6	12	88	82	18	54	84	37	0
4	89	30	79	69	87	10	74	4	56
29	40	12	19	22	37	49	14	92	54

Resultados de la suma:

81.00	91.00	13.00	95.00	137.00	62.00	97.00	106.00	47.00	66.00
141.00	6.00	18.00	57.00	176.00	126.00	80.00	110.00	107.00	55.00
143.00	133.00	127.00	125.00	74.00	87.00	86.00	129.00	148.00	133.00
54.00	109.00	39.00	127.00	121.00	189.00	65.00	136.00	157.00	31.00
81.00	76.00	177.00	36.00	122.00	77.00	92.00	142.00	103.00	143.00
83.00	84.00	149.00	38.00	77.00	100.00	87.00	89.00	111.00	50.00
126.00	141.00	112.00	148.00	150.00	68.00	53.00	77.00	171.00	4.00
85.00	70.00	78.00	167.00	150.00	20.00	76.00	157.00	107.00	32.00
92.00	152.00	52.00	145.00	78.00	168.00	52.00	133.00	93.00	117.00
81.00	114.00	96.00	43.00	72.00	57.00	75.00	40.00	171.00	148.00

Resultados de la resta:

-51.00	-67.00	5.00	-55.00	-27.00	-12.00	75.00	-42.00	43.00	-64.00
-41.00	6.00	12.00	-19.00	8.00	-20.00	-50.00	-74.00	31.00	9.00
29.00	-11.00	9.00	-35.00	48.00	-67.00	-80.00	-43.00	24.00	47.00
-30.00	-79.00	-27.00	-25.00	-47.00	9.00	15.00	-14.00	13.00	9.00
21.00	34.00	-19.00	-24.00	-70.00	69.00	-16.00	54.00	5.00	29.00
-19.00	8.00	3.00	-14.00	13.00	-24.00	-31.00	77.00	-67.00	-46.00
-24.00	-37.00	80.00	50.00	-20.00	-58.00	17.00	-77.00	11.00	2.00
71.00	58.00	54.00	-9.00	-14.00	-16.00	-32.00	-11.00	33.00	32.00
84.00	-26.00	-8.00	-13.00	-60.00	-6.00	32.00	-15.00	85.00	5.00
23.00	34.00	72.00	5.00	28.00	-17.00	-23.00	12.00	-13.00	40.00

Resultados de la multiplicación:

13243.00	17709.00	11624.00	16725.00	23176.00	15640.00
10412.00	19712.00	15740.00	8853.00		
13417.00	19626.00	17525.00	19138.00	25139.00	17263.00
14255.00	19554.00	16937.00	17637.00		
22813.00	29360.00	16956.00	32153.00	34711.00	29560.00
23210.00	35410.00	24280.00	24048.00		
15147.00	23004.00	17145.00	23933.00	28033.00	24630.00
16563.00	23605.00	22899.00	14685.00		
23875.00	25817.00	17541.00	31755.00	32049.00	29193.00
27284.00	34018.00	30087.00	20735.00		
17249.00	16545.00	14458.00	23930.00	24768.00	19256.00
19926.00	27028.00	18558.00	12295.00		
23009.00	33136.00	19352.00	32046.00	33834.00	32788.00
19876.00	36600.00	22228.00	18966.00		
23563.00	29750.00	17527.00	35298.00	39248.00	30120.00
22867.00	39719.00	23563.00	19891.00		
25656.00	32460.00	15594.00	33586.00	37685.00	34255.00
21625.00	35662.00	27847.00	21562.00		
23656.00	27483.00	16764.00	28865.00	30889.00	29851.00
24102.00	33348.00	26088.00	22294.00		

Transpuesta de la matriz A:

15.00	50.00	86.00	12.00	51.00	32.00	51.00	78.00	88.00	52.00
12.00	6.00	61.00	15.00	55.00	46.00	52.00	64.00	63.00	74.00
9.00	15.00	68.00	6.00	79.00	76.00	96.00	66.00	22.00	84.00
20.00	19.00	45.00	51.00	6.00	12.00	99.00	79.00	66.00	24.00
55.00	92.00	61.00	37.00	26.00	45.00	65.00	68.00	9.00	50.00
25.00	53.00	10.00	99.00	73.00	38.00	5.00	2.00	81.00	20.00
86.00	15.00	3.00	40.00	38.00	28.00	35.00	22.00	42.00	26.00
32.00	18.00	43.00	61.00	98.00	83.00	0.00	73.00	59.00	26.00
45.00	69.00	86.00	85.00	54.00	22.00	91.00	70.00	89.00	79.00
1.00	32.00	90.00	20.00	86.00	2.00	3.00	32.00	61.00	94.00

Transpuesta de la matriz B:

66.00	91.00	57.00	42.00	30.00	51.00	75.00	7.00	4.00	29.00
79.00	0.00	72.00	94.00	21.00	38.00	89.00	6.00	89.00	40.00
4.00	3.00	59.00	33.00	98.00	73.00	16.00	12.00	30.00	12.00
75.00	38.00	80.00	76.00	30.00	26.00	49.00	88.00	79.00	19.00
82.00	84.00	13.00	84.00	96.00	32.00	85.00	82.00	69.00	22.00
37.00	73.00	77.00	90.00	4.00	62.00	63.00	18.00	87.00	37.00
11.00	65.00	83.00	25.00	54.00	59.00	18.00	54.00	10.00	49.00
74.00	92.00	86.00	75.00	44.00	6.00	77.00	84.00	74.00	14.00
2.00	38.00	62.00	72.00	49.00	89.00	80.00	37.00	4.00	92.00
65.00	23.00	43.00	11.00	57.00	48.00	1.00	0.00	56.00	54.00

Inversa de la matriz A:

-0.60	-0.86	-4.88	-2.79	-4.32	-1.48	3.28	-1.78	0.42	-0.07
4.77	3.69	3.38	1.85	-4.80	3.48	-2.22	-3.48	0.22	0.60
3.78	-2.09	-2.78	-0.48	1.46	-2.62	4.78	0.01	-1.46	4.35
-2.09	-4.25	-4.46	1.49	-2.87	4.45	-1.09	3.81	2.16	2.90
1.09	-2.69	-4.89	4.60	-1.00	-3.74	1.68	-4.64	-0.92	0.48
-3.43	3.72	-0.07	1.35	-3.75	4.23	-0.36	-2.18	1.88	5.00
0.63	-2.95	-4.89	1.35	1.49	-4.14	2.54	4.37	4.79	3.55
-1.22	-0.63	1.07	-0.84	-2.88	-1.33	-4.86	1.76	2.99	3.24
2.93	-4.80	0.06	0.15	1.85	-4.20	-1.69	-1.29	-0.05	1.63
1.86	-3.44	-2.24	-1.45	2.94	2.39	-2.16	-0.89	-2.11	-4.20

Inversa de la matriz B:

-2.15	-1.53	-0.51	0.76	1.28	-0.98	-1.41	-0.39	0.44	-0.87
-0.16	-0.64	-1.79	2.63	-2.51	0.54	-0.63	-1.70	-1.31	2.35
-0.34	-1.23	-1.01	2.13	0.86	2.17	-0.71	-1.01	0.78	-0.90
-1.61	0.93	0.59	-0.98	1.58	2.16	1.15	1.56	0.92	-1.67
-1.81	0.84	-2.16	-2.03	-1.82	0.82	1.27	-0.41	0.26	-0.53
-1.09	2.47	2.52	-2.07	-0.55	2.33	1.05	-2.38	0.22	0.70
1.96	1.10	-2.03	1.63	-1.82	1.99	2.70	-1.90	-0.44	-1.49
-0.56	-2.54	-0.60	-0.38	-1.22	1.22	1.26	-0.68	-2.63	-2.09
-0.69	2.50	0.10	-0.27	-1.26	-0.12	-0.25	0.70	2.58	-0.07
1.93	-1.12	-0.24	-1.77	2.70	-1.30	-0.47	0.29	0.13	-1.56

said@said-VirtualBox:~/Descargas\$

WINDOWS TUBERIAS:

Padre.c

```
#include <windows.h>
#include <stdio.h>
#include <time.h>
#define N 10
#define OPERACIONES 5
void llenar(int matriz[N][N]) {
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            matriz[i][j] = rand() % 101;
        }
    }
}
void imprimir(int matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }
}
void imprimirInversa(double matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            printf("%lf ", matriz[i][j]);
        }
        printf("\n");
    }
}
int main(int argc, char *argv[]){
    int i, j;
    srand(time(NULL));
    int matrizA[N][N];
    int matrizB[N][N];
    llenar(matrizA);
    llenar(matrizB);
    printf("\nMatriz A: \n");
    imprimir(matrizA);
    printf("\nMatriz B: \n");
    imprimir(matrizB);
    HANDLE lecturaI, escrituraI;
    SECURITY_ATTRIBUTES pipeSeg = { sizeof(SECURITY_ATTRIBUTES), NULL, TRUE };
    DWORD escritos;
```



```

STARTUPINFO sHijo;
PROCESS_INFORMATION pHijo;
GetStartupInfo(&sHijo);
HANDLE lectura0, escritura0;
DWORD lectura;
sHijo.hStdError = GetStdHandle(STD_ERROR_HANDLE);
sHijo.dwFlags = STARTF_USESTDHANDLES;
int resultados[OPERACIONES][N][N];
double resultadosInv[2][N][N];
int indiceResultado = 0;
for (j = 1; j <= 6; j++){
    int resultado1[N][N], resultado2[N][N];
    double inversa1[N][N], inversa2[N][N];
    CreatePipe(&lecturaI, &escrituraI, &pipeSeg, 0);
    CreatePipe(&lectura0, &escritura0, &pipeSeg, 0);
    sHijo.hStdInput = lecturaI;
    sHijo.hStdOutput = escritura0;
    switch (j){
        case 1:
            CreateProcess(NULL, "sum.exe", NULL, NULL, TRUE, 0, NULL, NULL, &sHijo, &pHijo);
            WriteFile(escrituraI, matrizA, sizeof(matrizA), &escritos, NULL);
            WriteFile(escrituraI, matrizB, sizeof(matrizB), &escritos, NULL);
            ReadFile(lectura0, resultado1, sizeof(resultado1), &lectura, NULL);
            memcpy(resultados[indiceResultado++], resultado1, sizeof(resultado1));
            break;
        case 2:
            CreateProcess(NULL, "res.exe", NULL, NULL, TRUE, 0, NULL, NULL, &sHijo, &pHijo);
            WriteFile(escrituraI, matrizA, sizeof(matrizA), &escritos, NULL);
            WriteFile(escrituraI, matrizB, sizeof(matrizB), &escritos, NULL);
            ReadFile(lectura0, resultado1, sizeof(resultado1), &lectura, NULL);
            memcpy(resultados[indiceResultado++], resultado1, sizeof(resultado1));
            break;
        case 3:
            CreateProcess(NULL, "mul.exe", NULL, NULL, TRUE, 0, NULL, NULL, &sHijo, &pHijo);
            WriteFile(escrituraI, matrizA, sizeof(matrizA), &escritos, NULL);
            WriteFile(escrituraI, matrizB, sizeof(matrizB), &escritos, NULL);
            ReadFile(lectura0, resultado1, sizeof(resultado1), &lectura, NULL);
            memcpy(resultados[indiceResultado++], resultado1, sizeof(resultado1));
            break;
        case 4:
            CreateProcess(NULL, "tra.exe", NULL, NULL, TRUE, 0, NULL, NULL, &sHijo, &pHijo);
            WriteFile(escrituraI, matrizA, sizeof(matrizA), &escritos, NULL);
            WriteFile(escrituraI, matrizB, sizeof(matrizB), &escritos, NULL);
            ReadFile(lectura0, resultado1, sizeof(resultado1), &lectura, NULL);
            ReadFile(lectura0, resultado2, sizeof(resultado2), &lectura, NULL);
            memcpy(resultados[indiceResultado++], resultado1, sizeof(resultado1));
            memcpy(resultados[indiceResultado++], resultado2, sizeof(resultado2));
            break;
        case 5:
            CreateProcess(NULL, "inv.exe", NULL, NULL, TRUE, 0, NULL, NULL, &sHijo, &pHijo);
            WriteFile(escrituraI, matrizA, sizeof(matrizA), &escritos, NULL);
            WriteFile(escrituraI, matrizB, sizeof(matrizB), &escritos, NULL);
            ReadFile(lectura0, inversa1, sizeof(inversa1), &lectura, NULL);
            ReadFile(lectura0, inversa2, sizeof(inversa2), &lectura, NULL);
            memcpy(resultadosInv[0], inversa1, sizeof(inversa1));
            memcpy(resultadosInv[1], inversa2, sizeof(inversa2));
        case 6:
            sHijo.hStdOutput = GetStdHandle(STD_OUTPUT_HANDLE);
            CreateProcess(NULL, "col.exe", NULL, NULL, TRUE, 0, NULL, NULL, &sHijo, &pHijo);
            for(i = 0; i < indiceResultado; i++){
                WriteFile(escrituraI, resultados[i], sizeof(resultados[i]), &escritos, NULL);
            }
            WriteFile(escrituraI, resultadosInv[0], sizeof(resultadosInv[0]), &escritos, NULL);
            WriteFile(escrituraI, resultadosInv[1], sizeof(resultadosInv[1]), &escritos, NULL);
            WaitForSingleObject(pHijo.hProcess, INFINITE);
            break;
    }
}
CloseHandle(pHijo.hProcess);
CloseHandle(pHijo.hThread);
CloseHandle(lecturaI);
CloseHandle(escrituraI);
CloseHandle(lectura0);
CloseHandle(escritura0);
return 0;
}

```

Suma.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
void sumar(int A[N][N], int B[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = A[i][j] + B[i][j];
        }
    }
}
int main(int argc, char *argv[]){
    HANDLE tubLectura = GetStdHandle(STD_INPUT_HANDLE);
    DWORD lectura;
    HANDLE tubEscritura = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD escritura;
    int matrizA[N][N];
    int matrizB[N][N];
    ReadFile(tubLectura, matrizA, sizeof(matrizA), &lectura, NULL);
    ReadFile(tubLectura, matrizB, sizeof(matrizB), &lectura, NULL);
    int resultado[N][N];
    sumar(matrizA,matrizB,resultado);
    WriteFile(tubEscritura, resultado, sizeof(resultado), &escritura, NULL);
    CloseHandle(tubLectura);
    CloseHandle(tubEscritura);
    return 0;
}
```

Resta.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
void restar(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++) {
            resultado[i][j] = matrizuno[i][j] - matrizdos[i][j];
        }
    }
}
int main(int argc, char *argv[]){
    HANDLE tubLectura = GetStdHandle(STD_INPUT_HANDLE);
    DWORD lectura;
    HANDLE tubEscritura = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD escritura;
    int matrizA[N][N];
    int matrizB[N][N];
    ReadFile(tubLectura, matrizA, sizeof(matrizA), &lectura, NULL);
    ReadFile(tubLectura, matrizB, sizeof(matrizB), &lectura, NULL);
    int resultado[N][N];
    restar(matrizA,matrizB,resultado);
    WriteFile(tubEscritura, resultado, sizeof(resultado), &escritura, NULL);
    CloseHandle(tubLectura);
    CloseHandle(tubEscritura);
    return 0;
}
```

Multiplicación.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
void multiplicar(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++){
                resultado[i][j] += matrizuno[i][k] * matrizdos[k][j];
            }
        }
    }
}
int main(int argc, char *argv[]){
    HANDLE tubLectura = GetStdHandle(STD_INPUT_HANDLE);
    DWORD lectura;
    HANDLE tubEscritura = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD escritura;
    int matrizA[N][N];
    int matrizB[N][N];
    ReadFile(tubLectura, matrizA, sizeof(matrizA), &lectura, NULL);
    ReadFile(tubLectura, matrizB, sizeof(matrizB), &lectura, NULL);
    int resultado[N][N];
    multiplicar(matrizA,matrizB,resultado);
    WriteFile(tubEscritura, resultado, sizeof(resultado), &escritura, NULL);
    CloseHandle(tubLectura);
    CloseHandle(tubEscritura);
    return 0;
}
```

Transpuesta.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
void transponer(int matriz[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[j][i] = matriz[i][j];
        }
    }
}
int main(int argc, char *argv[]){
    HANDLE tubLectura = GetStdHandle(STD_INPUT_HANDLE);
    DWORD lectura;
    HANDLE tubEscritura = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD escritura;
    int matrizA[N][N];
    int matrizB[N][N];
    ReadFile(tubLectura, matrizA, sizeof(matrizA), &lectura, NULL);
    ReadFile(tubLectura, matrizB, sizeof(matrizB), &lectura, NULL);
    int traA[N][N];
    transponer(matrizA,traA);
    int traB[N][N];
    transponer(matrizB,traB);
    WriteFile(tubEscritura, traA, sizeof(traA), &escritura, NULL);
    WriteFile(tubEscritura, traB, sizeof(traB), &escritura, NULL);
    CloseHandle(tubLectura);
    CloseHandle(tubEscritura);
    return 0;
}
```

Inversa.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define N 10

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++){
        for (int col = 0; col < n; col++){
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int DeterminanteMatriz(int matriz[N][N], int n){
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];
    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++){
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}
```

```

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1){
        adjunta[0][0] = 1;
        return;
    }
    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

void inversa(int matriz[N][N], double inversa[N][N]){
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0){
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }
    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

```

```

int main(int argc, char *argv[]){
    HANDLE tubLectura = GetStdHandle(STD_INPUT_HANDLE);
    DWORD lectura;
    HANDLE tubEscritura = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD escritura;
    int matrizA[N][N];
    int matrizB[N][N];
    ReadFile(tubLectura, matrizA, sizeof(matrizA), &lectura, NULL);
    ReadFile(tubLectura, matrizB, sizeof(matrizB), &lectura, NULL);
    double invA[N][N];
    inversa(matrizA, invA);
    double invB[N][N];
    inversa(matrizB, invB);
    WriteFile(tubEscritura, invA, sizeof(invA), &escritura, NULL);
    WriteFile(tubEscritura, invB, sizeof(invB), &escritura, NULL);
    CloseHandle(tubLectura);
    CloseHandle(tubEscritura);
    return 0;
}

```

Colector.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define OPERACIONES 5

void imprimir(int matriz[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++){
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }
}

void ImprimirInversa(double matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            printf("%lf ", matriz[i][j]);
        }
        printf("\n");
    }
}

int main(int argc, char *argv[]){
    int i;
    int resultado[N][N];
    double inversa[N][N];
    DWORD lectura;
    char *nombres_operaciones[7] = {
        "suma", "resta", "multiplicacion", "traspuestaA", "traspuestaB", "inversaA", "inversaB"
    };
    for (i = 0; i < OPERACIONES; i++){
        ReadFile(GetStdHandle(STD_INPUT_HANDLE), resultado, sizeof(resultado), &lectura, NULL);
        printf("\n\nResultado de %s:\n", nombres_operaciones[i]);
        imprimir(resultado);
    }
    for (i = 5; i <= 6; i++){
        ReadFile(GetStdHandle(STD_INPUT_HANDLE), inversa, sizeof(inversa), &lectura, NULL);
        printf("\n\nResultado de %s:\n", nombres_operaciones[i]);
        ImprimirInversa(inversa);
    }
    return 0;
}
```

Matriz A:

13	71	18	62	20	1	57	99	37	43
59	86	100	48	35	80	8	80	20	9
92	6	75	85	2	78	32	61	43	65
85	15	79	12	16	79	24	75	11	77
11	38	86	88	65	44	51	31	7	82
92	49	62	20	79	3	78	81	74	9
91	64	20	59	71	24	29	15	3	12
43	67	35	23	32	2	76	80	8	89
45	44	88	65	14	60	83	72	44	4
89	39	63	23	23	62	42	34	34	96

Matriz B:

70	76	51	87	42	19	76	29	81	45
15	45	95	66	67	53	20	41	17	71
47	51	70	9	30	39	62	80	54	25
14	87	95	64	46	58	47	22	9	35
3	99	9	88	90	62	78	34	6	47
29	49	84	38	44	38	8	46	13	74
31	75	27	95	5	60	21	80	46	57
51	58	11	23	24	37	20	81	16	76
3	45	46	62	60	75	29	14	87	31
97	71	67	60	7	72	46	69	70	48

Resultado de suma:

83	147	69	149	62	20	133	128	118	88
74	131	195	114	102	133	28	121	37	80
139	57	145	94	32	117	94	141	97	90
99	102	174	76	62	137	71	97	20	112
14	137	95	176	155	106	129	65	13	129
121	98	146	58	123	41	86	127	87	83
122	139	47	154	76	84	50	95	49	69
94	125	46	46	56	39	96	161	24	165
48	89	134	127	74	135	112	86	131	35
186	110	130	83	30	134	88	103	104	144

Resultado de resta:

-57	-5	-33	-25	-22	-18	-19	70	-44	-2
44	41	5	-18	-32	27	-12	39	3	-62
45	-45	5	76	-28	39	-30	-19	-11	40
71	-72	-16	-52	-30	21	-23	53	2	42
8	-61	77	0	-25	-18	-27	-3	1	35
63	0	-22	-18	35	-35	70	35	61	-65
60	-11	-7	-36	66	-36	8	-65	-43	-45
-8	9	24	0	8	-35	56	-1	-8	13
42	-1	42	3	-46	-15	54	58	-43	-27
-8	-32	-4	-37	16	-10	-4	-35	-36	48

Resultado de multiplicacion:

14876	27259	22033	24311	15721	22540	14234	22882	14358	23244
18478	31794	32393	25281	23341	23161	20792	27184	17341	28094
24050	34990	33025	28664	18697	25091	22666	27028	24134	26195
24466	29775	26348	23581	15532	20765	19967	27304	21288	24988
19222	34939	30366	28175	19415	27019	22152	27274	17126	24150
18337	36584	23554	34543	24006	26810	24760	27359	24278	27132
12842	28186	22271	27263	19639	18021	19487	16084	13483	19779
21229	30354	22912	27738	14774	23655	17893	27293	18639	24956
17403	32534	29301	27792	19179	24769	19026	28179	19672	26481
24415	32516	29758	29376	17835	24571	21785	26574	24579	25463

Resultado de traspuestaA:

13	59	92	85	11	92	91	43	45	89
71	86	6	15	38	49	64	67	44	39
18	100	75	79	86	62	20	35	88	63
62	48	85	12	88	20	59	23	65	23
20	35	2	16	65	79	71	32	14	23
1	80	78	79	44	3	24	2	60	62
57	8	32	24	51	78	29	76	83	42
99	80	61	75	31	81	15	80	72	34
37	20	43	11	7	74	3	8	44	34
43	9	65	77	82	9	12	89	4	96

Resultado de traspuestaB:

70	15	47	14	3	29	31	51	3	97
76	45	51	87	99	49	75	58	45	71
51	95	70	95	9	84	27	11	46	67
87	66	9	64	88	38	95	23	62	60
42	67	30	46	90	44	5	24	60	7
19	53	39	58	62	38	60	37	75	72
76	20	62	47	78	8	21	20	29	46
29	41	80	22	34	46	80	81	14	69
81	17	54	9	6	13	46	16	87	70
45	71	25	35	47	74	57	76	31	48

Resultado de inversaA:

-1.303686	0.736432	-1.257338	1.250513	-1.635773	-1.120512	0.289883	0.323478	0.481905	0.639374
0.988656	1.218031	-0.096056	0.130233	0.619478	-0.158090	0.395483	-0.851201	0.842189	0.137532
1.113538	1.253375	1.570754	1.266847	1.099333	0.656433	-0.707196	0.894122	-1.596332	0.522693
0.004601	0.977821	1.223821	-0.559116	0.203505	-0.628004	-1.531611	-1.505135	1.403782	-0.619079
0.658217	-1.030452	1.032742	0.303403	0.841307	-0.318763	-1.525133	-0.749287	1.580712	1.672059
-0.394823	1.283685	-0.216809	1.679381	-1.022332	-0.217764	-0.272437	1.060657	0.200652	0.423393
0.434492	-0.303798	1.396887	-0.983484	-0.910081	0.278882	0.608222	1.560685	-0.660568	1.034975
-0.079496	1.343607	-1.038716	1.669820	0.144640	1.077061	-0.129157	0.183027	-0.411364	0.652489
0.599025	0.734506	1.046771	-0.226011	-0.982733	0.621201	-0.936505	-0.900143	-0.186427	0.150957
1.719352	0.241217	0.355381	0.150353	0.959649	0.751188	0.510618	-0.715861	1.183383	0.184170

Resultado de inversaB:

27.801426	24.069577	11.569946	-9.823950	-13.147719	-5.337876	11.291256	-0.974681	18.588197	6.840223
3.713547	-12.045257	14.865828	-22.502434	-28.246124	19.852859	-1.810115	14.078948	-9.966823	-26.938372
-21.244764	-24.635791	7.704317	0.008975	-1.032853	16.257480	2.974586	-10.451243	0.022888	13.945210
22.068125	4.391678	-8.927115	-13.426252	-20.523009	19.425894	17.384320	5.733390	2.412623	-24.159160
2.928801	11.055155	-21.632122	-22.110210	20.257159	21.228806	13.000839	14.119069	14.203488	-2.082295
17.174024	6.054121	-27.901566	15.393316	-4.303676	20.420145	-11.503638	16.261363	-16.362348	-0.606418
-15.629735	-17.654876	-9.869973	10.200571	27.740356	-5.207287	28.464593	0.703505	-12.856628	26.008391
9.724832	15.466316	8.889602	-6.839266	5.737977	12.324715	14.419801	2.521720	6.330975	24.338008
15.048600	5.965621	-2.893405	5.392105	16.230994	22.631452	-24.515518	-4.507743	2.549072	-13.006646
5.002160	-24.255113	-16.889869	-25.524596	-10.672726	-1.170990	-25.087090	-26.100379	12.748756	-10.083829

WINDOWS COMPARTIDA

Padre.c

```
#include <windows.h>
#include <stdio.h>
#include <time.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))
void LlenarMatriz(int matriz[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = rand() % 101;
        }
    }
}
void ImprimirMatriz(int matriz[N][N]){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }
}
HANDLE CrearMemoriaCompartida(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, TAM_MEM, nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}
void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}
```

```

int main(int argc, char *argv[]) {
    srand(time(NULL));
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    LlenarMatriz(Matrizuno);
    LlenarMatriz(Matrizdos);
    printf("Matriz A:\n");
    ImprimirMatriz(Matrizuno);
    printf("\nMatriz B:\n");
    ImprimirMatriz(Matrizdos);
    STARTUPINFO si[6];
    PROCESS_INFORMATION pi[6];
    HANDLE handles[6];
    for (int j = 1; j <= 6; j++){
        ZeroMemory(&si[j-1], sizeof(si[j-1]));
        si[j-1].cb = sizeof(si[j-1]);
        ZeroMemory(&pi[j-1], sizeof(pi[j-1]));
        int* apDatosUno;
        int* apDatosDos;
        HANDLE hMemComUno = CrearMemoriaCompartida("Memoria_Compartida_MU_PH", &apDatosUno);
        HANDLE hMemComDos = CrearMemoriaCompartida("Memoria_Compartida_MD_PH", &apDatosDos);
        EscribirMemoriaCompartida(Matrizuno, apDatosUno);
        EscribirMemoriaCompartida(Matrizdos, apDatosDos);
        switch (j){
            case 1:
                if(!CreateProcess(NULL, "sum", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
                    printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
                    return 1;
                }
                while (apDatosUno[0] != -1 && apDatosDos[0] != -1){
                    Sleep(1);
                }
                break;
            case 2:
                if(!CreateProcess(NULL, "res", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
                    printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
                    return 1;
                }
        }
    }
}

```

```

while (apDatosUno[0] != -1 && apDatosDos[0] != -1){
    Sleep(1);
}
break;
case 3:
if(!CreateProcess(NULL, "mul", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
    printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
    return 1;
}
while (apDatosUno[0] != -1 && apDatosDos[0] != -1){
    Sleep(1);
}
break;
case 4:
if(!CreateProcess(NULL, "tra", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
    printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
    return 1;
}
while (apDatosUno[0] != -1 && apDatosDos[0] != -1){
    Sleep(1);
}
break;
case 5:
if(!CreateProcess(NULL, "inv", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
    printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
    return 1;
}
while (apDatosUno[0] != -1 && apDatosDos[0] != -1){
    Sleep(1);
}
break;
case 6:
if(!CreateProcess(NULL, "col", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
    printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
    return 1;
}
break;

```

```

    }
    handles[j-1] = pi[j-1].hProcess;
    UnmapViewOfFile(apDatosUno);
    UnmapViewOfFile(apDatosDos);
    CloseHandle(hMemComUno);
    CloseHandle(hMemComDos);
}
WaitForMultipleObjects(6, handles, TRUE, INFINITE);
for (int j = 0; j < 6; j++) {
    CloseHandle(pi[j].hProcess);
    CloseHandle(pi[j].hThread);
}
return 0;
}

```

Suma.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))
void SumarMatrices(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = matrizuno[i][j] + matrizdos[i][j];
        }
    }
}
HANDLE CrearMemoriaCompartida(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, TAM_MEM, nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}
void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}
void ErrorMemoriaCompartida(const char* nombre, HANDLE* hMemCom){
    if ((*hMemCom = OpenFileMapping(
        FILE_MAP_ALL_ACCESS,
        FALSE,
        nombre
    )) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
}
```

```

int* MapearMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((apDatos = (int *)MapViewOfFile(
        hMemCom,
        FILE_MAP_ALL_ACCESS,
        0,
        0,
        TAM_MEM
    )) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE hMemComMatrizUno;
    int* apDatosMatrizUno;
    ErrorMemoriaCompartida("Memoria_Compartida_MU_PH", &hMemComMatrizUno);
    apDatosMatrizUno = MapearMemoriaCompartida("Memoria_Compartida_MU_PH", hMemComMatrizUno);
    LeerMemoriaCompartida(Matrizuno, apDatosMatrizUno);
    apDatosMatrizUno[0] = -1;
    UnmapViewOfFile(apDatosMatrizUno);
    CloseHandle(hMemComMatrizUno);
    HANDLE hMemComMatrizDos;
    int* apDatosMatrizDos;
    ErrorMemoriaCompartida("Memoria_Compartida_MD_PH", &hMemComMatrizDos);
    apDatosMatrizDos = MapearMemoriaCompartida("Memoria_Compartida_MD_PH", hMemComMatrizDos);
    LeerMemoriaCompartida(Matrizdos, apDatosMatrizDos);
    apDatosMatrizDos[0] = -1;
    UnmapViewOfFile(apDatosMatrizDos);
    CloseHandle(hMemComMatrizDos);

    int Resultado[N][N];
    SumarMatrices(Matrizuno, Matrizdos, Resultado);
    int* apDatosResum;
    HANDLE hMemComResum = CrearMemoriaCompartida("Memoria_Compartida_Res_Sum", &apDatosResum);
    EscribirMemoriaCompartida(Resultado, apDatosResum);
    while (apDatosResum[0] != -1) {
        Sleep(1);
    }
    UnmapViewOfFile(apDatosResum);
    CloseHandle(hMemComResum);
    return 0;
}

```

Resta.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))

void RestarMatrices(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++) {
            resultado[i][j] = matrizuno[i][j] - matrizdos[i][j];
        }
    }
}

HANDLE CrearMemoriaCompartida(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, TAM_MEM, nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void ErrorMemoriaCompartida(const char* nombre, HANDLE* hMemCom){
    if ((*hMemCom = OpenFileMapping(
        FILE_MAP_ALL_ACCESS,
        FALSE,
        nombre
    )) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
}
```

```

int* MapearMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((apDatos = (int *)MapViewOfFile(
        hMemCom,
        FILE_MAP_ALL_ACCESS,
        0,
        0,
        TAM_MEM
    )) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE hMemComMatrizUno;
    int* apDatosMatrizUno;
    ErrorMemoriaCompartida("Memoria_Compartida_MU_PH", &hMemComMatrizUno);
    apDatosMatrizUno = MapearMemoriaCompartida("Memoria_Compartida_MU_PH", hMemComMatrizUno);
    LeerMemoriaCompartida(Matrizuno, apDatosMatrizUno);
    apDatosMatrizUno[0] = -1;
    UnmapViewOfFile(apDatosMatrizUno);
    CloseHandle(hMemComMatrizUno);
    HANDLE hMemComMatrizDos;
    int* apDatosMatrizDos;
    ErrorMemoriaCompartida("Memoria_Compartida_MD_PH", &hMemComMatrizDos);
    apDatosMatrizDos = MapearMemoriaCompartida("Memoria_Compartida_MD_PH", hMemComMatrizDos);
    LeerMemoriaCompartida(Matrizdos, apDatosMatrizDos);
    apDatosMatrizDos[0] = -1;
    UnmapViewOfFile(apDatosMatrizDos);
    CloseHandle(hMemComMatrizDos);

    int Resultado[N][N];
    RestarMatrices(Matrizuno, Matrizdos, Resultado);
    int* apDatosResrest;
    HANDLE hMemComResrest = CrearMemoriaCompartida("Memoria_Compartida_Res_Rest", &apDatosResrest);
    EscribirMemoriaCompartida(Resultado, apDatosResrest);
    while (apDatosResrest[0] != -1) {
        Sleep(1);
    }
    UnmapViewOfFile(apDatosResrest);
    CloseHandle(hMemComResrest);
    return 0;
}

```

Multiplicación.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))

void MultiplicarMatrices(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++){
                resultado[i][j] += matrizuno[i][k] * matrizdos[k][j];
            }
        }
    }
}

HANDLE CrearMemoriaCompartida(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, TAM_MEM, nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void ErrorMemoriaCompartida(const char* nombre, HANDLE* hMemCom){
    if ((*hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE, nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
}
```



```

int* MapearMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((apDatos = (int *)MapViewOfFile(hMemCom, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE hMemComMatrizUno;
    int* apDatosMatrizUno;
    ErrorMemoriaCompartida("Memoria_Compartida_MU_PH", &hMemComMatrizUno);
    apDatosMatrizUno = MapearMemoriaCompartida("Memoria_Compartida_MU_PH", hMemComMatrizUno);
    LeerMemoriaCompartida(Matrizuno, apDatosMatrizUno);
    apDatosMatrizUno[0] = -1;
    UnmapViewOfFile(apDatosMatrizUno);
    CloseHandle(hMemComMatrizUno);
    HANDLE hMemComMatrizDos;
    int* apDatosMatrizDos;
    ErrorMemoriaCompartida("Memoria_Compartida_MD_PH", &hMemComMatrizDos);
    apDatosMatrizDos = MapearMemoriaCompartida("Memoria_Compartida_MD_PH", hMemComMatrizDos);
    LeerMemoriaCompartida(Matrizdos, apDatosMatrizDos);
    apDatosMatrizDos[0] = -1;
    UnmapViewOfFile(apDatosMatrizDos);
    CloseHandle(hMemComMatrizDos);
    int Resultado[N][N];
    MultiplicarMatrices(Matrizuno, Matrizdos, Resultado);
    int* apDatosResmul;
    HANDLE hMemComResmul = CrearMemoriaCompartida("Memoria_Compartida_Res_Mul", &apDatosResmul);
    EscribirMemoriaCompartida(Resultado, apDatosResmul);

    while (apDatosResmul[0] != -1) {
        Sleep(1);
    }
    UnmapViewOfFile(apDatosResmul);
    CloseHandle(hMemComResmul);
    return 0;
}

```

Transpuesta.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))
void TransponerMatriz(int matriz[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[j][i] = matriz[i][j];
        }
    }
}
HANDLE CrearMemoriaCompartida(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, TAM_MEM, nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}
void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}
void ErrorMemoriaCompartida(const char* nombre, HANDLE* hMemCom){
    if ((*hMemCom = OpenFileMapping(
        FILE_MAP_ALL_ACCESS,
        FALSE,
        nombre
    )) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
}
```

```

int* MapearMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((apDatos = (int *)MapViewOfFile(hMemCom, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE hMemComMatrizUno;
    int* apDatosMatrizUno;
    ErrorMemoriaCompartida("Memoria_Compartida_MU_PH", &hMemComMatrizUno);
    apDatosMatrizUno = MapearMemoriaCompartida("Memoria_Compartida_MU_PH", hMemComMatrizUno);
    LeerMemoriaCompartida(Matrizuno, apDatosMatrizUno);
    apDatosMatrizUno[0] = -1;
    UnmapViewOfFile(apDatosMatrizUno);
    CloseHandle(hMemComMatrizUno);
    HANDLE hMemComMatrizDos;
    int* apDatosMatrizDos;
    ErrorMemoriaCompartida("Memoria_Compartida_MD_PH", &hMemComMatrizDos);
    apDatosMatrizDos = MapearMemoriaCompartida("Memoria_Compartida_MD_PH", hMemComMatrizDos);
    LeerMemoriaCompartida(Matrizdos, apDatosMatrizDos);
    apDatosMatrizDos[0] = -1;
    UnmapViewOfFile(apDatosMatrizDos);
    CloseHandle(hMemComMatrizDos);
    int Traspuesta_uno[N][N];
    TransponerMatriz(Matrizuno, Traspuesta_uno);
    int* apDatosRestrasu;
    HANDLE hMemComRestrasu = CrearMemoriaCompartida("Memoria_Compartida_Res_TrasU", &apDatosRestrasu);
    EscribirMemoriaCompartida(Traspuesta_uno, apDatosRestrasu);

    while (apDatosRestrasu[0] != -1) {
        Sleep(1);
    }
    UnmapViewOfFile(apDatosRestrasu);
    CloseHandle(hMemComRestrasu);
    int Traspuesta_dos[N][N];
    TransponerMatriz(Matrizdos, Traspuesta_dos);
    int* apDatosRestrasd;
    HANDLE hMemComRestrasd = CrearMemoriaCompartida("Memoria_Compartida_Res_TrasD", &apDatosRestrasd);
    EscribirMemoriaCompartida(Traspuesta_dos, apDatosRestrasd);
    while (apDatosRestrasd[0] != -1) {
        Sleep(1);
    }
    UnmapViewOfFile(apDatosRestrasd);
    CloseHandle(hMemComRestrasd);
    return 0;
}

```

Inversa.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM_INT (N * N * sizeof(int))
#define TAM_MEM_DOUBLE (N * N * sizeof(double))
void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++){
        for (int col = 0; col < n; col++){
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1){
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int DeterminanteMatriz(int matriz[N][N], int n){
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];
    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++){
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}
```

```

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1){
        adjunta[0][0] = 1;
        return;
    }
    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

void InversaMatriz(int matriz[N][N], double inversa[N][N]){
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }
    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

HANDLE crearMemoria(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, TAM_MEM_INT, nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM_INT)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}

```

```

HANDLE crearMemoriaD(const char* nombre, double** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, TAM_MEM_DOUBLE, nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (double*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM_DOUBLE)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}

void escribirMemoria(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void escribirMemoriaD(double matriz[N][N], double* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void errorMem(const char* nombre, HANDLE* hMemCom){
    if ((*hMemCom = OpenFileMapping(
        FILE_MAP_ALL_ACCESS,
        FALSE,
        nombre
    )) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
}

```

```

int* mapearMemoria(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((apDatos = (int *)MapViewOfFile(hMemCom, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM_INT)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void leerMemoria(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

int main(void){
    int matrizA[N][N];
    int matrizB[N][N];
    HANDLE memMata;
    int* apDatosMata;
    errorMem("Memoria Compartida_MU_PH", &memMata);
    apDatosMata = mapearMemoria("Memoria Compartida_MU_PH", memMata);
    leerMemoria(matrizA, apDatosMata);
    apDatosMata[0] = -1;
    UnmapViewOfFile(apDatosMata);
    CloseHandle(memMata);
    HANDLE memMatB;
    int* apDatosMatB;
    errorMem("Memoria Compartida_MD_PH", &memMatB);
    apDatosMatB = mapearMemoria("Memoria Compartida_MD_PH", memMatB);
    leerMemoria(matrizB, apDatosMatB);
    apDatosMatB[0] = -1;
    UnmapViewOfFile(apDatosMatB);
    CloseHandle(memMatB);
    double inversaA[N][N];
    InversaMatriz(matrizA, inversaA);
    double* apDatosResinvu;
    HANDLE memResinvu = crearMemoriaD("Memoria Compartida_Res_InvU", &apDatosResinvu);
    escribirMemoriaD(inversaA, apDatosResinvu);

    while (apDatosResinvu[0] != -1) {
        Sleep(1);
    }
    UnmapViewOfFile(apDatosResinvu);
    CloseHandle(memResinvu);
    double inversaB[N][N];
    InversaMatriz(matrizB, inversaB);
    double* apDatosResinvd;
    HANDLE hMemComResinvd = crearMemoriaD("Memoria Compartida_Res_InvD", &apDatosResinvd);
    escribirMemoriaD(inversaB, apDatosResinvd);
    while (apDatosResinvd[0] != -1) {
        Sleep(1);
    }
    UnmapViewOfFile(apDatosResinvd);
    CloseHandle(hMemComResinvd);
    return 0;
}

```

Colector.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 10
#define TAM_MEM_INT (N * N * sizeof(int))
#define TAM_MEM_DOUBLE (N * N * sizeof(double))
void ImprimirMatriz(int matriz[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }
}
void ImprimirInversa(double matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            printf("%lf ", matriz[i][j]);
        }
        printf("\n");
    }
}
void EsperarMemoriaCompartida(const char* nombre, HANDLE* hMemCom){
    while ((*hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE,nombre)) == NULL) {
        Sleep(100);
    }
}
void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}
void LeerMemoriaCompartidaD(double matriz[N][N], double* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}
```



```

int* MapearMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((apDatos = (int *)MapViewOfFile(hMemCom,FILE_MAP_ALL_ACCESS, 0,0,TAM_MEM_INT)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

double* MapearMemoriaCompartidaD(const char* nombre, HANDLE hMemCom){
    double* apDatos;
    if ((apDatos = (double *)MapViewOfFile(hMemCom,FILE_MAP_ALL_ACCESS, 0,0,TAM_MEM_DOUBLE)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

int main(int argc, char *argv[]){
    HANDLE hMemComResum;
    int* apDatosResum;
    EsperarMemoriaCompartida("Memoria_Compartida_Res_Sum", &hMemComResum);
    apDatosResum = MapearMemoriaCompartida("Memoria_Compartida_Res_Sum", hMemComResum);
    int Resultado_sum[N][N];
    LeerMemoriaCompartida(Resultado_sum, apDatosResum);
    printf("\nResultado suma: \n");
    ImprimirMatriz(Resultado_sum);
    apDatosResum[0] = -1;
    UnmapViewOfFile(apDatosResum);
    CloseHandle(hMemComResum);
    HANDLE hMemComResrest;
    int* apDatosResrest;
    EsperarMemoriaCompartida("Memoria_Compartida_Res_Rest", &hMemComResrest);
    apDatosResrest = MapearMemoriaCompartida("Memoria_Compartida_Res_Rest", hMemComResrest);
    int Resultado_rest[N][N];
    LeerMemoriaCompartida(Resultado_rest, apDatosResrest);
    printf("\nResultado resta: \n");
    ImprimirMatriz(Resultado_rest);
    apDatosResrest[0] = -1;
    UnmapViewOfFile(apDatosResrest);
    CloseHandle(hMemComResrest);
}

```

```

HANDLE hMemComResmul;
int* apDatosResmul;
EsperarMemoriaCompartida("Memoria_Compartida_Res_Mul", &hMemComResmul);
apDatosResmul = MapearMemoriaCompartida("Memoria_Compartida_Res_Mul", hMemComResmul);
int Resultado_multi[N][N];
LeerMemoriaCompartida(Resultado_multi, apDatosResmul);
printf("\nResultado multiplicacion: \n");
ImprimirMatriz(Resultado_multi);
apDatosResmul[0] = -1;
UnmapViewOfFile(apDatosResmul);
CloseHandle(hMemComResmul);
HANDLE hMemComRestrasu;
int* apDatosRestrasu;
EsperarMemoriaCompartida("Memoria_Compartida_Res_TrasU", &hMemComRestrasu);
apDatosRestrasu = MapearMemoriaCompartida("Memoria_Compartida_Res_TrasU", hMemComRestrasu);
int Resultado_traspuestau[N][N];
LeerMemoriaCompartida(Resultado_traspuestau, apDatosRestrasu);
printf("\nResultado traspuesta uno: \n");
ImprimirMatriz(Resultado_traspuestau);
apDatosRestrasu[0] = -1;
UnmapViewOfFile(apDatosRestrasu);
CloseHandle(hMemComRestrasu);
HANDLE hMemComRestrasd;
int* apDatosRestrasd;
EsperarMemoriaCompartida("Memoria_Compartida_Res_TrasD", &hMemComRestrasd);
apDatosRestrasd = MapearMemoriaCompartida("Memoria_Compartida_Res_TrasD", hMemComRestrasd);
int Resultado_traspuestad[N][N];
printf("\nResultado traspuesta dos: \n");
LeerMemoriaCompartida(Resultado_traspuestad, apDatosRestrasd);
ImprimirMatriz(Resultado_traspuestad);
apDatosRestrasd[0] = -1;
UnmapViewOfFile(apDatosRestrasd);
CloseHandle(hMemComRestrasd);
HANDLE hMemComResinvu;
double* apDatosResinvu;
EsperarMemoriaCompartida("Memoria_Compartida_Res_InvU", &hMemComResinvu);
apDatosResinvu = MapearMemoriaCompartidaD("Memoria_Compartida_Res_InvU", hMemComResinvu);
double Resultado_inversau[N][N];
printf("\nResultado inversa uno: \n");
LeerMemoriaCompartidaD(Resultado_inversau, apDatosResinvu);

```

```

ImprimirInversa(Resultado_inversau);
apDatosResinvu[0] = -1;
UnmapViewOfFile(apDatosResinvu);
CloseHandle(hMemComResinvu);
HANDLE hMemComResinvd;
double* apDatosResinvd;
EsperarMemoriaCompartida("Memoria_Compartida_Res_InvD", &hMemComResinvd);
apDatosResinvd = MapearMemoriaCompartidaD("Memoria_Compartida_Res_InvD", hMemComResinvd);
double Resultado_inversad[N][N];
printf("\nResultado inversa dos: \n");
LeerMemoriaCompartidaD(Resultado_inversad, apDatosResinvd);
ImprimirInversa(Resultado_inversad);
apDatosResinvd[0] = -1;
UnmapViewOfFile(apDatosResinvd);
CloseHandle(hMemComResinvd);
return 0;
}

```

C:\Users\vbcsl>padre

Matriz A:

51	70	91	3	14	46	71	30	34	76
32	52	50	70	35	44	68	32	21	77
41	33	28	30	23	34	0	32	34	91
26	24	86	28	43	17	97	57	47	19
39	68	0	49	48	57	72	97	57	55
50	67	81	4	73	38	3	61	64	94
38	79	91	0	31	83	82	5	86	0
31	76	27	26	77	2	86	33	9	45
27	93	72	7	8	10	31	17	1	30
31	100	57	63	88	74	87	83	87	70

Matriz B:

8	48	89	68	4	26	85	65	18	92
76	20	88	86	20	67	86	35	43	83
51	85	11	18	87	17	10	53	71	15
99	60	74	40	65	58	23	77	61	48
80	41	17	44	37	78	41	29	70	62
73	84	80	1	70	4	45	95	33	75
32	71	13	25	15	73	56	16	26	80
1	64	86	69	92	76	47	49	83	24
70	72	32	85	14	5	31	86	31	6
64	98	83	29	43	44	48	39	45	80

Resultado suma:

59	118	180	71	18	72	156	95	52	168
108	72	138	156	55	111	154	67	64	160
92	118	39	48	110	51	10	85	105	106
125	84	160	68	108	75	120	134	108	67
119	109	17	93	85	135	113	126	127	117
123	151	161	5	143	42	48	156	97	169
70	150	104	25	46	156	138	21	112	80
32	140	113	95	169	78	133	82	92	69
97	165	104	92	22	15	62	103	32	36
95	198	140	92	131	118	135	122	132	150

Resultado resta:

43	22	2	-65	10	20	-14	-35	16	-16
-44	32	-38	-16	15	-23	-18	-3	-22	-6
-10	-52	17	12	-64	17	-10	-21	-37	76
-73	-36	12	-12	-22	-41	74	-20	-14	-29
-41	27	-17	5	11	-21	31	68	-13	-7
-23	-17	1	3	3	34	-42	-34	31	19
6	8	78	-25	16	10	26	-11	60	-80
30	12	-59	-43	-15	-74	39	-16	-74	21
-43	21	40	-78	-6	5	0	-69	-30	24
-33	2	-26	34	45	30	39	44	42	-10

Resultado multiplicacion:

24690	33058	26739	20847	21023	19990	24066	24089	21880	29013
28306	32091	27968	19858	22012	23021	22376	24600	23074	29334
19792	24021	23585	16113	15774	14029	16692	19552	16512	20599
21538	29690	18779	19313	20841	21276	18786	21234	22450	21889
28243	33742	34124	27562	23634	27717	27821	28447	26021	31665
29286	34987	30949	26480	24803	22555	25148	28130	27495	28552
28137	31716	22750	22168	19500	17374	23433	27795	20664	26761
22576	23464	21116	20057	15059	24690	22092	16053	20064	27063
16018	17165	17220	14675	13016	14107	15958	13132	14512	18159
42871	46550	41127	35527	32783	35463	35022	38640	36348	41111

Resultado traspuesta uno:

51	32	41	26	39	50	38	31	27	31
70	52	33	24	68	67	79	76	93	100
91	50	28	86	0	81	91	27	72	57
3	70	30	28	49	4	0	26	7	63
14	35	23	43	48	73	31	77	8	88
46	44	34	17	57	38	83	2	10	74
71	68	0	97	72	3	82	86	31	87
30	32	32	57	97	61	5	33	17	83
34	21	34	47	57	64	86	9	1	87
76	77	91	19	55	94	0	45	30	70

Resultado traspuesta dos:

8	76	51	99	80	73	32	1	70	64
48	20	85	60	41	84	71	64	72	98
89	88	11	74	17	80	13	86	32	83
68	86	18	40	44	1	25	69	85	29
4	20	87	65	37	70	15	92	14	43
26	67	17	58	78	4	73	76	5	44
85	86	10	23	41	45	56	47	31	48
65	35	53	77	29	95	16	49	86	39
18	43	71	61	70	33	26	83	31	45
92	83	15	48	62	75	80	24	6	80

Resultado inversa uno:

0.247197	-1.093225	-0.347294	0.531311	0.308095	0.329630	-0.126155	0.489786	0.783445	0.881354
1.169502	-1.172831	-1.035519	0.969252	1.140840	-0.690445	-0.251235	1.071212	-1.109049	0.483738
-0.542721	-1.135905	-0.392283	-0.557649	-1.140202	0.291435	-0.352248	-1.141884	-0.788192	-0.026403
0.048094	-0.241412	-0.978272	-0.706142	0.920728	-0.914582	-0.105653	0.354184	-1.129746	-0.588725
-0.802423	-0.439011	0.421323	1.133837	1.123198	-1.111010	-0.379508	-0.330041	-1.052711	1.065157
-0.935132	1.154034	-0.948782	-0.823456	-0.069808	-0.409733	0.656825	-1.040409	0.739053	-0.328680
0.166899	0.246466	0.453404	-0.342838	-0.411265	1.054805	0.559587	0.712667	0.416223	0.314536
-0.505443	-0.929889	-0.556605	1.135732	-0.423621	0.035210	0.190219	-0.774901	0.206313	0.059120
0.425371	0.558344	0.597096	0.166576	-1.075406	0.269452	-0.973059	-0.297372	0.763956	0.904420
0.109057	-0.458766	0.172063	-0.497669	-0.082920	-0.348488	0.127913	0.377713	-0.324737	-0.771946

Resultado inversa dos:

7.930792	1.060848	4.235808	-0.433663	6.236653	0.070505	-1.397613	3.846551	-3.652983	-7.678180
-6.183012	-7.284645	-3.670357	-8.038193	2.174368	2.712767	5.506503	3.914928	7.154627	-1.749125
-4.598553	4.860941	-1.096150	-7.099515	4.183583	5.747231	-6.448517	5.050977	-0.763057	-1.146975
6.297244	-1.124388	-6.342818	-6.636541	7.541570	2.556768	5.277439	7.623123	2.760129	-5.023448
-3.647234	7.308578	0.626106	7.695403	5.472005	-3.391828	1.030534	0.130891	-0.001359	4.865476
-4.992310	2.879569	6.323762	5.544954	6.572433	-7.030888	-1.356712	3.494618	-0.103894	2.272336
2.855211	7.555000	-4.848274	-0.214894	6.985714	-7.423618	-4.758814	-1.079738	6.679276	-6.778548
-5.819926	-2.732813	-1.956967	6.732235	-6.126301	-1.267567	-5.014877	-1.946798	-2.767718	4.495293
0.072121	7.982787	-0.219367	0.578937	0.403800	-7.546425	7.414836	1.948919	5.919205	-6.522761
0.637605	7.169583	-1.936561	2.244940	2.281683	-1.712873	1.680892	0.371203	6.766804	1.497933

Conclusión

En esta práctica se abordaron los principales mecanismos de comunicación entre procesos (IPC) en Linux y Windows, explorando tanto las tuberías como la memoria compartida para coordinar la transferencia de datos entre procesos relacionados. Las tareas incluyeron la implementación de operaciones matemáticas sobre matrices y la gestión de jerarquías de procesos, destacando el intercambio eficiente de información entre ellos. En el caso de las tuberías, se utilizó un flujo de datos secuencial que permitió enviar mensajes y estructuras, como matrices, desde un proceso padre a sus hijos y viceversa. Este mecanismo, aunque simple y efectivo, es unidireccional y requiere un diseño cuidadoso para sincronizar los accesos de lectura y escritura. Se verificó su funcionamiento tanto en Linux, con la función `pipe()`, como en Windows, mediante `CreatePipe()`.

La memoria compartida, por otro lado, facilitó un acceso más directo y rápido a los datos al permitir que múltiples procesos trabajaran en una misma región de memoria. En Linux, se implementó utilizando `shmget()` y `shmat()`, mientras que en Windows se emplearon funciones como `CreateFileMapping()` y `MapViewOfFile()`. Este enfoque redujo la necesidad de copiar grandes estructuras entre procesos, como matrices de 10x10, pero implicó mayor complejidad al manejar concurrencia y sincronización, ya que todos los procesos deben respetar el acceso coordinado para evitar colisiones.

En las tareas específicas, se desarrolló un escenario donde un proceso padre enviaba matrices a un hijo para su multiplicación, y este, a su vez, delegaba la suma de matrices a un nieto, utilizando IPC para transferir los resultados. Una vez obtenidos los resultados, el proceso padre realizaba operaciones finales, como calcular las matrices inversas, y almacenaba los resultados en archivos. Estas operaciones resaltaron la importancia de sincronizar procesos, gestionar correctamente los recursos compartidos y manejar adecuadamente las jerarquías de procesos.

Comparativamente, la implementación en Linux y Windows permitió observar diferencias significativas en la API y el manejo de recursos. Linux presentó una mayor simplicidad en el diseño de IPC, mientras que Windows ofreció un control más detallado a través de estructuras de seguridad y configuraciones avanzadas. Sin embargo, ambos sistemas requerían un manejo riguroso de errores para garantizar la estabilidad de la comunicación y evitar problemas como accesos concurrentes no controlados o fugas de recursos.

En general, esta práctica proporcionó un entendimiento integral de los conceptos y herramientas de IPC, fortaleciendo habilidades tanto en programación de sistemas como en desarrollo multiplataforma. También evidenció la importancia de elegir el mecanismo de comunicación adecuado según las características del problema, el tamaño de los datos a transferir y los requisitos de sincronización entre procesos.